



**Politecnico
di Torino**

Politecnico di Torino

Ingegneria Informatica

A.a. 2025/2026

Sessione di laurea Luglio 2026

Realizzazione di un workflow di elaborazione dati con lineage end-to-end

Relatori:

Marco Torchiano

Candidato:

Mattia Domizio

Ringraziamenti

Desidero ringraziare il professor Torchiano, i miei genitori Claudia e Massimiliano per avermi permesso di intraprendere questo percorso, la mia fidanzata Gaia per avermi sempre supportato e sopportato in questi anni di studio, il mio tutor Edgal per avermi guidato in questa esperienza, i colleghi che ho conosciuto in azienda e tutti gli amici che mi sono stati accanto durante questa fase della mia vita.

Indice

Elenco delle figure	VIII
1 Introduzione	1
1.1 Contesto e motivazioni	2
1.2 DataHub Core	3
1.3 Agenti AI e server MCP	4
1.4 Obiettivo della tesi	5
1.5 Struttura della tesi	6
2 Architettura di Teseo	8
2.1 Requisiti architetturali	9
2.2 Principi di progettazione adottati	10
2.3 Stack infrastrutturale di DataHub Core	11
2.3.1 Struttura generale dell'architettura	11
2.3.2 Client Tier	13
2.3.3 App Tier	13
2.3.4 Persistence Tier	13
2.3.5 Percorso interno dei metadati	14
2.4 Modello e grafo dei metadati	14
2.4.1 Entità, URN e aspetti	15
2.4.2 Entity Registry e governabilità del modello	15
2.4.3 Relazioni e lettura del lineage	16
2.4.4 Versionamento e coerenza temporale	17
2.4.5 Ciclo degli eventi metadata (MCP e MCL)	18
2.5 Ingestion layer multi-source	18
2.5.1 Metodi di integrazione supportati	19
2.5.2 Pattern implementativo in Teseo	21
2.6 Exposure Layer: UI, GraphQL e API	21
2.6.1 UI per navigazione	22
2.6.2 GraphQL per interrogazione	22
2.6.3 SDK e API per automazione	22

2.7	Sicurezza e osservabilità	22
2.8	Lifecycle del metadato nell'architettura Teseo	23
2.9	Scenari di rischio e strategie di mitigazione	24
2.9.1	Rischio di incompletezza del lineage	24
2.9.2	Rischio di deriva semantica	25
2.9.3	Rischio di regressione	25
2.10	Benefici architetturali osservati	25
2.11	Posizionamento dei moduli evolutivi	26
3	Ingestion multi-source e lineage end-to-end	27
3.1	Snowflake: DDL e setup	27
3.1.1	Setup iniziale e validazione permessi	28
3.1.2	Deployment DDL del dominio e delle procedure	29
3.2	Configurazione DataHub	36
3.3	Ingestion Snowflake	36
3.4	Ingestion dbt e raccordo con Snowflake	39
3.4.1	Setup progetto dbt e generazione artefatti	39
3.4.2	Recipe dbt e controllo del rumore nel grafo	40
3.4.3	Riconciliazione bridge Snowflake-dbt	41
3.5	Ingestion Power BI: da mock locale a service	43
3.5.1	Motivazione del modello locale	43
3.5.2	Script di generazione e ingestion	44
3.5.3	Connessione a Power BI Service	45
3.6	Costruzione del lineage end-to-end	46
3.7	Esempio di ingestion da altre sorgenti	48
3.7.1	Caso Databricks (Unity Catalog)	48
3.7.2	Il vero valore di Teseo	49
4	Data Governance	50
4.1	Data Governance e metadati	50
4.1.1	Cos'è la Data Governance	50
4.1.2	Dimensioni principali della governance	51
4.2	Ruolo della governance in Teseo	52
4.3	Entità di Governance in Teseo	52
4.3.1	Tagging e classificazione	52
4.3.2	Domini e ownership	53
4.3.3	Business Glossary e Structured Properties	54
4.3.4	Assertions e Data Contracts	54
4.3.5	Incident management	55

5	MCP Server e strumenti AI	57
5.1	Model Context Protocol	57
5.1.1	Data layer e transport layer	58
5.1.2	Lifecycle di sessione	59
5.1.3	Sicurezza e trust boundaries	60
5.1.4	MCP in contesti data-centric	61
5.2	DataHub MCP Server	61
5.2.1	Deployment	61
5.2.2	Catalogo degli strumenti	62
5.3	Strumenti write-oriented custom	62
5.3.1	Struttura del file	63
5.3.2	Merge non distruttivo	63
5.3.3	Tipologie di tool	64
5.3.4	Dettaglio operativo delle famiglie di tool	65
5.4	Tool Power BI	66
5.4.1	Flusso operativo	66
5.4.2	Generazione di un progetto PBIP	67
5.4.3	Validazione, stima e packaging	67
5.4.4	Reverse engineering e conversione tra formati	68
5.4.5	Implicazioni per Teseo	69
5.5	Metriche di efficacia e adozione in produzione	70
5.5.1	KPI tecnici e di governance	70
5.5.2	Modello di adozione del progetto Teseo	71
5.5.3	Osservabilità e Interpretazione dei KPI	71
5.6	Limiti	72
6	Risultati e valutazioni	74
6.1	Dimensione del problema e della soluzione	74
6.1.1	Dimensione del problema	74
6.1.2	Dimensione della soluzione	75
6.2	Valutazione dell'ingestion multi-source	76
6.2.1	Copertura del catalogo	76
6.2.2	Tempi di ingestion e idempotenza	77
6.2.3	Lineage end-to-end	78
6.3	Valutazione della governance dei metadati	79
6.3.1	Arricchimento semantico e coverage	79
6.3.2	Qualità e monitoraggio degli asset	80
6.4	Valutazione degli strumenti MCP	80
6.4.1	KPI tecnici dell'agente	80
6.4.2	KPI di governance	81
6.5	Valutazione della pipeline Power BI	82

6.6	Analisi complessiva degli obiettivi raggiunti	82
6.7	Limiti emersi nella valutazione	84
7	Conclusioni	85
7.1	Contributi della tesi	85
7.2	Direzioni evolutive	86
7.3	Considerazioni finali	87
	Bibliografia	90

Elenco delle figure

1.1	Dati e Metadati	2
1.2	Layer di DataHub Core	4
2.1	Gestione stream-based dei metadati tramite Kafka	10
2.2	Architettura a tre livelli di DataHub Core nel deployment di Teseo .	12
2.3	Metadata Model semplificato	16
2.4	Flusso di metadata ingestion	19
2.5	Pubblicazione sincrona e asincrona di un MCP	20
2.6	Sezione di Teseo per la gestione dei token	23
3.1	Struttura database e schemi in Snowflake	35
3.2	Risultato lineage dopo ingestion Snowflake + dbt	42
3.3	Lineage end-to-end in Teseo	47
4.1	Data Governance in Teseo	51
4.2	Ciclo di vita di un incident generato da un'assertion fallita	56
5.1	Architettura MCP in Teseo	58
5.2	Data layer e transport layer	59
5.3	Flusso di pubblicazione Power BI da Teseo	69
6.1	Stateful ingestion e soft delete degli asset obsoleti	77

Capitolo 1

Introduzione

La crescente complessità dei data warehouse aziendali rende sempre più difficile comprendere in modo affidabile *da dove provengono i dati, come vengono trasformati e dove vengono utilizzati*. Nella pratica, molte organizzazioni utilizzano contemporaneamente piattaforme cloud, strumenti ELT, sistemi di BI e pipeline orchestrate da tool differenti. In assenza di un catalogo tecnologico unificato, questa eterogeneità produce frammentazione informativa, duplicazione semantica e scarsa tracciabilità end-to-end [1].

Il problema non riguarda solo la documentazione, ma la capacità operativa di gestire il rischio: senza lineage affidabile, un cambiamento su una tabella sorgente può propagarsi a valle senza visibilità immediata, impattando KPI, dashboard e processi decisionali. In parallelo, aumentano le richieste di auditabilità, compliance e data governance, che richiedono una ricostruzione trasparente delle dipendenze tra layer tecnologici differenti [2]. Proprio in questo scenario si inserisce **Teseo**, iniziativa orientata allo sviluppo di una piattaforma di **metadata ingestion multi-source** e **lineage e2e**, basata su **DataHub Core** (open-source, versione 1.4.0) e progressivamente estesa con componenti AI costruiti su server MCP.

La tesi nasce da un'attività progettuale in corso, svolta in azienda, e documenta lo stato di avanzamento, le decisioni tecniche e i risultati ottenuti. Nella prima fase del lavoro, il focus è stato posto su due direttrici principali: *(i)* consolidamento della base documentale e metodologica su DataHub Core; *(ii)* progettazione e implementazione del flusso di metadata ingestion **Snowflake + dbt**, esigenza prioritaria per estendere le integrazioni già presenti (BigQuery, Qlik e Databricks) [3, 4, 5]. Dopo aver ottenuto i risultati attesi su questi fronti, la tesi si è concentrata sull'integrazione con **Power BI** e sul raffinamento del lineage end-to-end mediante modelli AI costruiti su **server MCP** [6, 7].

1.1 Contesto e motivazioni

Le organizzazioni che adottano stack dati eterogenei affrontano frequentemente problemi di frammentazione: cataloghi incompleti, ownership non esplicita, lineage parziale e difficoltà nel valutare gli impatti a valle dovuti a cambiamenti sui dataset a monte.

In questo contesto, la **metadata ingestion** non è un'attività accessoria ma un elemento infrastrutturale: consente di estrarre, normalizzare e pubblicare metadati tecnici e semantici provenienti da sistemi diversi, rendendo possibile una vista coerente del catalogo dati. Tale processo riduce il tempo necessario per la comprensione delle dipendenze, facilita la collaborazione tra team e supporta decisioni più rapide in presenza di incident o modifiche evolutive.

Il lavoro è stato svolto presso **Target Reply**, società del gruppo Reply specializzata nello sviluppo di soluzioni avanzate in ambito *Analytics*, *Big Data*, *Data Science*, *Cloud Architectures* e *Intelligenza Artificiale* [8]. In questo contesto professionale, Teseo si colloca all'interno di un percorso end-to-end che va dall'identificazione dei bisogni informativi alla progettazione e realizzazione di soluzioni di integrazione, modellazione e valorizzazione del dato.

Con il termine **metadati** si indicano le informazioni descrittive che contestualizzano il dato, offrendo una comprensione più profonda del contenuto e delle sue caratteristiche. Come illustrato nella Figura 1.1, attorno a un dato (tabella o colonna) si possono distinguere quattro dimensioni di metadati: *struttura* (schema, tipi, chiavi...), *semantica* di business (definizioni, domini, glossario...), *responsabilità* organizzative (ownership, compliance...) e *tracce operative* (origine, frequenza, quality...) [1]. In altri termini, i metadati trasformano un insieme di tabelle senza significato in un patrimonio informativo interpretabile, governabile e riusabile.

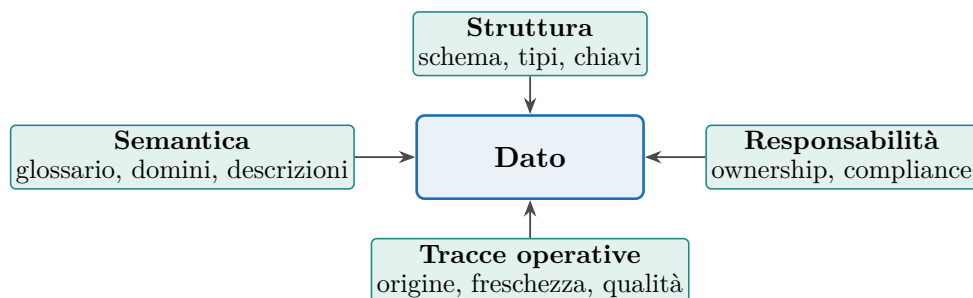


Figura 1.1: Dati e Metadati

All'interno di questa cornice, il **lineage** rappresenta la *dimensione dinamica* del metadato, in quanto descrive i legami di derivazione delle entità e permette di ricostruire il percorso che compiono i dati lungo i diversi layer, fino all'effettiva origine del dato stesso [2]. Questo abilita l'*impact analysis*, la *root-cause analysis* e la *validazione* delle trasformazioni applicate nel tempo.

In questo scenario, il contributo di **Teseo** è rendere questi elementi osservabili in modo unificato anche in presenza di sorgenti eterogenee, non solo mostrando il catalogo completo degli asset, ma anche il lineage end-to-end. Questo garantisce continuità semantica e tracciabilità del flusso dati, requisiti essenziali per **affidabilità** analitica e **governance** operativa.

1.2 DataHub Core

DataHub Core è stato adottato come backbone del progetto Teseo per la sua capacità di integrare in un unico grafo di conoscenza metadati provenienti da piattaforme differenti e per la sua natura *open-source* [9]. La scelta è stata guidata anche dalla disponibilità di connettori maturi, API estendibili e un modello orientato alla gestione evolutiva delle piattaforme di metadati. In termini di funzionalità, la piattaforma abilita:

- catalogazione unificata di dataset, viste, pipeline, dashboard e relativi attributi;
- ricerca e discovery su asset eterogenei;
- rappresentazione del lineage a livello tabellare e, dove disponibile, a livello di colonna;
- supporto alla governance (ownership, descrizioni, domini, tagging e quality).

L'approccio che abbiamo deciso di adottare nel progetto Teseo è stato **progressivo**:

- *consolidamento dell'ingestion e del lineage su sorgenti prioritarie*: in questa fase, abbiamo focalizzato l'attenzione su Snowflake (piattaforma di data warehouse ampiamente adottata dai clienti) e su dbt (strumento di orchestrazione che consente di costruire un lineage dettagliato a livello di modello e di gestire complessi processi di trasformazione); la scelta di dbt è stata guidata anche dalla sua capacità di integrarsi perfettamente con Snowflake [3, 4]. L'obiettivo è stato quindi garantire una base solida di metadati tecnici e semantici, con un lineage affidabile e facilmente navigabile;

- *estensione verso livelli di consumo analitico*: dopo aver consolidato la base tecnica, abbiamo esteso la copertura verso strumenti di business intelligence come Power BI, con l'obiettivo di abilitare un lineage end-to-end che arrivi fino al livello più vicino al customer [6]. Questo passaggio è stato cruciale per rendere i metadati realmente utili agli utenti finali, facilitando la comprensione delle dipendenze e della impact analysis tramite strumenti AI-powered (e non solo).

La Figura 1.2 sintetizza il ruolo di DataHub come layer unificante: i connettori raccolgono i metadati dalle diverse sorgenti, l'Ingestion Framework li normalizza e li pubblica verso il Metadata Service, che li rende persistenti e interrogabili attraverso un unico catalogo.

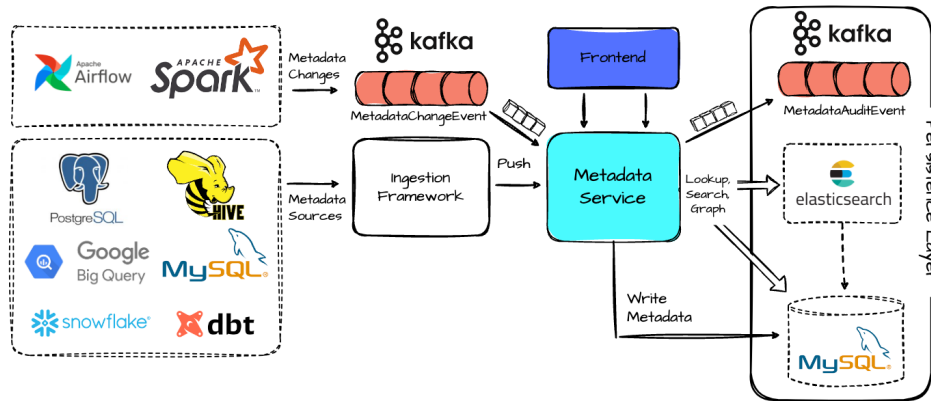


Figura 1.2: Layer di DataHub Core

Fonte: documentazione ufficiale di DataHub [10]

1.3 Agenti AI e server MCP

Oltre alla pura catalogazione dei metadati, il progetto prevede l'adozione di agenti AI in grado di interrogare e agire sul catalogo metadati in modo controllato. In questo scenario, il **Model Context Protocol (MCP)** svolge un ruolo centrale in quanto fornisce un'interfaccia standard attraverso cui i modelli possono accedere a strumenti e contesti esterni, mantenendo separazione tra logica dell'agente e sorgenti dei metadati [7, 11].

Nel caso di DataHub Core, l'utilizzo di un server MCP consente di costruire agenti orientati a casi d'uso concreti di data governance, come ad esempio:

- ricerca contestuale di dataset e proprietari (*ownership discovery*);

- navigazione del lineage upstream/downstream;
- analisi dell'impatto (*impact analysis*) a valle di una modifica su colonne e dipendenze eseguita a monte;
- assistenza tecnica nella comprensione delle varie trasformazioni applicate ai campi.

Questo approccio permette di integrare le potenzialità dei modelli AI con la ricchezza informativa del catalogo metadati, mantenendo però un controllo rigoroso sulle operazioni eseguite e sulla tracciabilità delle interazioni. In particolare, l'adozione del protocollo MCP consente di introdurre *guardrail* applicativi e una chiara separazione delle responsabilità tra livello conversazionale, livello di orchestrazione e livello dati.

1.4 Obiettivo della tesi

L'obiettivo generale della tesi è descrivere progettazione, implementazione e validazione progressiva della piattaforma Teseo, con particolare attenzione a:

- adozione di DataHub Core 1.4.0 come nucleo del metadata platform;
- implementazione del connettore Snowflake e relativa pipeline di ingestion;
- integrazione di dbt per abilitare lineage model-to-model e contesto semantico;
- definizione di una base architetturale estendibile verso funzionalità AI;
- preparazione dell'integrazione con strumenti BI (in particolare Power BI) per un lineage end-to-end più leggibile e utile al business.

Oltre agli aspetti tecnici, la tesi intende mostrare in modo concreto quali criticità aziendali vengono risolte dall'adozione di Teseo e quali benefici operativi ne derivano. In particolare, la piattaforma è progettata per intervenire su quattro problemi ricorrenti nelle organizzazioni data-driven:

- *scarsa visibilità del patrimonio dati*: i metadati risultano spesso distribuiti tra strumenti diversi e difficili da consultare in modo uniforme;
- *lineage incompleto o frammentato*: senza una vista end-to-end delle dipendenze, l'impact analysis di una modifica resta parziale e rischiosa;
- *governance non sistematica*: ownership, domini, classificazioni e policy risultano frequentemente non standardizzati;

- *bassa produttività nelle attività di analisi*: data engineer, analyst e stakeholder spendono tempo eccessivo nel ricostruire contesto, origine e significato dei dati.

In questa prospettiva, Teseo non viene trattato semplicemente come un enorme catalogo, ma come un **facilitatore organizzativo** capace di migliorare qualità decisionale, collaborazione e velocità di esecuzione. I principali vantaggi attesi sono:

- riduzione dei tempi di *impact analysis* e *root-cause analysis*;
- incremento dell'affidabilità dei flussi analitici grazie a metadati più completi e coerenti;
- maggiore trasparenza verso funzioni business e governance tramite tassonomie condivise (tag, domini, ownership);
- accelerazione dell'onboarding tecnico su nuove sorgenti e nuovi progetti;
- evoluzione verso un modello operativo assistito da agenti AI, con interazioni controllate tramite MCP.

Un ulteriore obiettivo riguarda quindi l'adozione del MCP server per abilitare agenti AI capaci di interrogare e manipolare metadati in modo controllato, mantenendo tracciabilità, sicurezza e governance delle operazioni.

1.5 Struttura della tesi

Nei capitoli successivi verranno esposti i principali step di sviluppo che hanno accompagnato questo percorso di Tesi, seguendo la struttura illustrata di seguito:

- **Capitolo 2 - Architettura di Teseo.** Questo capitolo illustra nel dettaglio la struttura architeturale della piattaforma, descrivendo componenti, responsabilità, flussi di comunicazione e ruolo di DataHub Core come nucleo di Teseo.
- **Capitolo 3 - Ingestion multi-source.** In questo capitolo viene analizzata l'implementazione e il funzionamento dei flussi di ingestion, con particolare attenzione su Snowflake, dbt e Power BI, evidenziando come tali integrazioni costruiscano un lineage completo end-to-end; il capitolo include inoltre altri esempi di ingestion da sorgenti come Databricks o PostgreSQL.

- **Capitolo 4 - Data Governance con Teseo/DataHub Core.** Questo capitolo approfondisce l'utilizzo delle funzionalità di governance (tag, domini, ownership, documentazione, classificazione degli asset...), mostrando come Teseo supporti controllo e standardizzazione del patrimonio informativo.
- **Capitolo 5 - MCP Server e strumenti AI.** In questo capitolo si descrive l'implementazione e l'utilizzo del server MCP per abilitare operazioni assistite da agenti AI su DataHub Core/Teseo, con focus su casi d'uso pratici e benefici in termini di produttività.
- **Capitolo 6 - Risultati e valutazioni.** Questa sezione presenta una valutazione complessiva dei risultati ottenuti, misurando il guadagno operativo dell'adozione di Teseo e il suo posizionamento all'interno dei processi aziendali, con particolare attenzione alle organizzazioni data-driven.
- **Capitolo 7 - Conclusione.** Questo capitolo conclusivo riassume i contributi della tesi, i limiti attuali della soluzione e le principali direttrici evolutive del progetto.

Capitolo 2

Architettura di Teseo

Teseo è stato concepito e implementato come **piattaforma di metadata intelligence** basata su **DataHub Core**, con l'obiettivo di fornire una vista unificata, affidabile e navigabile del catalogo dati aziendale. In questo capitolo viene descritta l'architettura di Teseo, evidenziando componenti, responsabilità e flussi operativi che consentono di passare da un insieme eterogeneo di sorgenti di dati a un catalogo coerente e interrogabile in modo sistematico [10, 12].

Come già brevemente annunciato in precedenza, l'architettura adottata è stata così definita per rispondere principalmente a quattro esigenze concrete emerse durante il lavoro progettuale:

- **integrazione multi-source dei metadati** al fine di superare la frammentazione informativa e costruire un catalogo centralizzato;
- **tracciabilità end-to-end delle dipendenze** per abilitare impact analysis più affidabili e ridurre la dipendenza da conoscenza tacita;
- **supporto operativo a data engineer e data analyst** attraverso un'interfaccia user-friendly e strumenti di navigazione del grafo dei metadati (*search function*);
- **estendibilità verso nuove funzionalità**, senza però compromettere la stabilità del nucleo della piattaforma.

Da ciò ne è pertanto derivata una separazione netta tra livello di raccolta dei metadati, livello di gestione del grafo dei metadati, livello di consultazione dei metadati e del lineage, e livello di integrazione con strumenti esterni.

2.1 Requisiti architetturali

La soluzione è stata sviluppata in un **contesto enterprise** in cui coesistono sistemi di **data warehouse**, strumenti di **ETL** e piattaforme **Business Intelligence**, ma ognuno di questi ha logiche operative differenti. Pertanto, in assenza di una piattaforma centrale che fa da *facilitatore* e *orchestratore* dei metadati, la comprensione delle dipendenze passate tra tutti questi layer risulta frammentata: infatti lo stesso dataset può essere descritto:

- in modo diverso a seconda dello **strumento** utilizzato e del **livello operativo** (*layer*) in cui ci troviamo;
- con **ownership** non sempre esplicita e che può variare a seconda del **contesto** (ad esempio: ownership tecnica su un job di trasformazione, ownership funzionale su un dataset, ownership di consumo su una dashboard...);
- con **lineage** spesso incompleto (limitato a pochi layer o a pochi asset all'interno di un layer) e non sempre aggiornato in modo affidabile e stabile.

Per affrontare nelle migliori condizioni questo complesso scenario, l'architettura di Teseo è stata guidata dai seguenti **requisiti architetturali**:

- **Unificazione semantica**: rappresentare asset eterogenei (dataset, dashboard, job, policy, ownership) secondo un modello unificato;
- **Tracciabilità tecnica**: mantenere ovunque relazioni upstream/downstream e informazioni di contesto utili all'impact analysis, alla data discovery e alla navigazione del lineage;
- **Operatività**: consentire ingestion schedulata o on-demand con configurazione esplicita e verificabile, mediante le **ricette YAML** (*metadata ingestion recipe*);
- **Scalabilità**: sostenere un numero crescente di sorgenti senza riscrivere il nucleo applicativo, ma semplicemente estendendolo con nuove ricette e nuovi **connettori**;
- **Estendibilità**: integrare API e strumenti esterni, seppur preservando la coerenza del metadata model.

A seguito di una valutazione approfondita, DataHub Core è risultato il componente più adatto a soddisfare tali requisiti grazie a diverse caratteristiche:

- **modello schema-first**: prima si definisce un modello ben tipizzato dei metadati (*JSON Schema* con tipi, campi, relazioni...) e poi si configura l'ingestion per popolare questo modello. Grazie a questo *contratto chiaro* tra modello e ingestion, è possibile mantenere coerenza tra sorgenti diverse, ridurre il rischio di ambiguità semantica e facilitare l'estensione progressiva e controllata del catalogo [9];
- **gestione stream-based** dei cambiamenti sui metadati: con l'adozione di un broker Kafka per la propagazione degli eventi sui metadati, DataHub consente di mantenere sincronizzazione tra stato transazionale e stato consultabile, riducendo il rischio di disallineamento tra catalogo visualizzato e metadati effettivamente ingestiti. La Figura 2.1 illustra questo flusso: una proposta di modifica (*Metadata Change Proposal*) viene propagata tramite un topic Kafka al Metadata Service, il quale aggiorna lo stato transazionale (MySQL) e, attraverso il log di modifica (*Metadata Change Log*), lo stato consultabile e indicizzato (ElasticSearch);

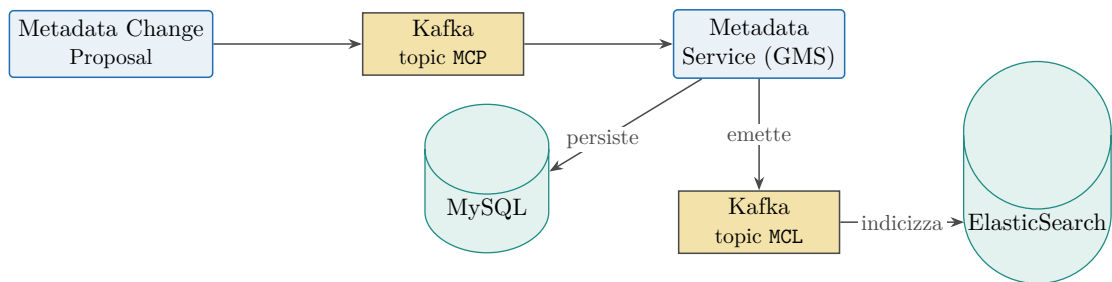


Figura 2.1: Gestione stream-based dei metadati tramite Kafka

- **disponibilità di API**: mediante DataHub Core, si può disporre di API orientate sia all'interazione umana (*UI* e *GraphQL*) sia all'automazione (*Python SDK*, *Java SDK* e API di basso livello come *OpenAPI*) [13].

2.2 Principi di progettazione adottati

La definizione architetturale di Teseo non si è limitata alla scelta dei componenti e al soddisfacimento dei requisiti, ma ha seguito anche un insieme di principi che hanno guidato l'implementazione:

- **Separazione delle responsabilità**: la raccolta dei metadati, la loro persistenza e la loro esposizione sono state mantenute su piani distinti;
- **Configurabilità esplicita**: ogni integrazione è stata descritta tramite ricette YAML versionabili, riducendo dipendenze implicite;

- **Evoluzione incrementale:** prima consolidamento del core stack e delle sorgenti prioritarie (in questo caso specifico Snowflake), poi estensione progressiva ad altre sorgenti e piattaforme;
- **Aderenza al prodotto open-source:** le customizzazioni sono state contenute per evitare di divergere troppo da DataHub Core, in modo da facilitare aggiornamenti futuri e mantenere compatibilità con le nuove versioni.

Questi principi hanno consentito di costruire una piattaforma robusta e flessibile, idonea a sostenere data discovery, governance avanzata e strumenti AI.

2.3 Stack infrastrutturale di DataHub Core

Come spiegato prima, Teseo è stato implementato in ambiente on-premises utilizzando il quickstart Docker di DataHub 1.4.0, ma opportunamente configurato sui requisiti del progetto: in particolare, è stato personalizzato il frontend React per renderlo adatto all'uso di strumenti AI-powered, ma è stata anche abilitata l'autenticazione mediante l'opzione `METADATA_SERVICE_AUTH_ENABLED` impostata a `true` nel `docker-compose.yml`. Questa scelta ha permesso di ottenere un ambiente riproducibile, rapido da avviare e coerente con la documentazione ufficiale [12, 10].

2.3.1 Struttura generale dell'architettura

L'architettura di DataHub Core si può suddividere in 3 tiers principali:

- **Client Tier;**
- **App Tier;**
- **Persistence Tier.**

La Figura 2.2 mostra i tre tier dell'architettura adottata in Teseo, riportando nel Client Tier le sole sorgenti effettivamente integrate nel progetto (Snowflake, dbt e Power BI). Nel deployment utilizzato, i servizi sono configurati con autenticazione abilitata lato metadata service, in modo da garantire accesso controllato alle operazioni di lettura e modifica, inclusa la generazione di token personali per integrazioni esterne. Tutto questo avviene grazie alla variabile `METADATA_SERVICE_AUTH_ENABLED` impostata a `true` nel `docker-compose.yml`. Vediamo di seguito questi tre tiers più nel dettaglio.

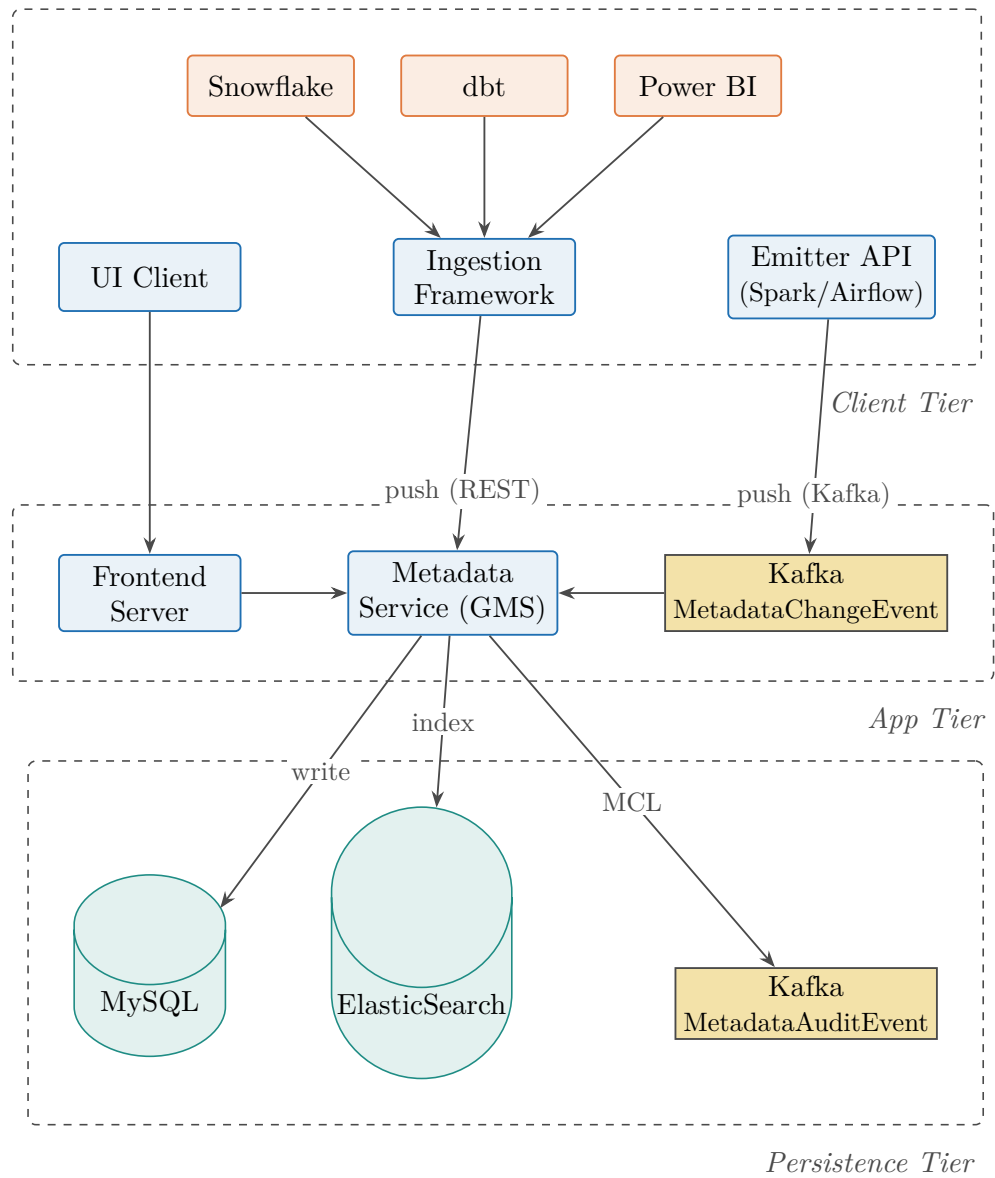


Figura 2.2: Architettura a tre livelli di DataHub Core nel deployment di Teseo

2.3.2 Client Tier

Il **Client Tier** include i canali di interazione con gli utenti e con le ingestion dalle sorgenti esterne. In particolare, comprende:

- **UI Client:** interfaccia utente principale per data discovery, navigazione del grafo e gestione operativa. Comunica con il Frontend Server dell'App Tier tramite API REST e GraphQL;
- **Ingestion Framework:** strumento di raccolta e normalizzazione dei metadati ricevuti dalle sorgenti (come BigQuery, Snowflake, PostgreSQL, Databricks, dbt, Looker, MySQL...), configurabile tramite ricette YAML o direttamente dalla UI. Dialoga con il Metadata Service dell'App Tier per la persistenza dei metadati e con il Kafka broker dell'App Tier per la propagazione degli eventi di cambiamento;
- **Emitter API:** subisce il real-time push da parte di servizi come Apache Spark e Apache Airflow, e si occupa di inoltrare gli eventi al Kafka broker dell'App Tier (*MetadataChangeEvent*).

2.3.3 App Tier

L'**App Tier** include i servizi applicativi che gestiscono la logica di business e l'interazione con i componenti del Persistence Tier. In particolare, comprende:

- **Frontend Server:** gestisce le richieste provenienti dalla UI Client, interagendo con il Metadata Service per leggere e modificare i metadati;
- **Metadata Service:** gestisce la logica di persistenza dei metadati, ricevendo le richieste dal Frontend Server e dagli strumenti di ingestion, e interagendo con i componenti del Persistence Tier per salvare e indicizzare i dati;
- **Kafka broker** (*MetadataChangeEvent*): gestisce la propagazione degli eventi di cambiamento sui metadati, consentendo di mantenere sincronizzazione tra stato transazionale e stato consultabile, riducendo il rischio di disallineamento tra catalogo visualizzato e metadati effettivamente ingestiti.

2.3.4 Persistence Tier

Il **Persistence Tier** include i componenti responsabili della persistenza e dell'indicizzazione dei metadati. In particolare, comprende:

- **MySQL**: database relazionale che funge da archivio primario per i metadati, gestendo la persistenza degli aspetti e delle relazioni ricevuti dal Metadata Service;
- **Elasticsearch**: gestisce la logica di indicizzazione dei metadati, ricevendo le richieste dal Metadata Service e creando indici ottimizzati per la ricerca ed il metadata discovery;
- **Kafka broker** (*MetadataAuditEvent*): gestisce la propagazione degli eventi di audit sui metadati, consentendo di mantenere tracciabilità e conformità.

2.3.5 Percorso interno dei metadati

Come si può vedere dall'architettura sopra descritta, le sorgenti esterne non scrivono direttamente sul catalogo, bensì inviano metadati attraverso l'Ingestion Framework, che li mappa sul metadata model (*schema-first model*) e li inoltra agli endpoint di input della piattaforma. Da qui, la catena di persistenza e indicizzazione aggiorna lo stato del grafo dei metadati e la rappresentazione interrogabile direttamente dalla UI.

Infatti, il percorso interno può essere riassunto in quattro passaggi:

1. estrazione e normalizzazione dei metadati dalla sorgente;
2. emissione verso DataHub con validazione dello schema;
3. persistenza e aggiornamento delle relazioni;
4. indicizzazione per ricerca, discovery e lineage navigation.

Questa pipeline è ciò che rende possibile mantenere coerenza tra stato transazionale e stato consultabile, riducendo (come detto in precedenza) il rischio di disallineamento tra catalogo visualizzato e metadati ingestiti.

2.4 Modello e grafo dei metadati

Uno dei fattori determinanti nella progettazione di Teseo è stato il **Metadata Model** di DataHub, basato su entità, aspetti e relazioni. Tale modello consente di descrivere in modo omogeneo asset differenti e di collegarli lungo dimensioni tecniche e semantiche [14]. In questa sezione si approfondiscono i meccanismi che rendono il modello realmente estendibile al trascorrere del tempo: *Entity Registry*, *key aspects*, tipologie di aspetti e ciclo dei *metadata events*.

2.4.1 Entità, URN e aspetti

Ogni asset è identificato da una **URN** (*Uniform Resource Name*) stabile e appartiene a una specifica classe di entità (ad esempio dataset, data job, dashboard). Le proprietà dell'entità non sono trattate come blocco unico, ma suddivise in aspetti, ciascuno dedicato a una specifica prospettiva informativa (schema, ownership, descrizioni, tag, lineage...).

Questa struttura offre due vantaggi architetturali rilevanti:

- **evoluzione non distruttiva**: si possono introdurre nuovi aspetti senza ridefinire l'intera entità;
- **aggiornamento selettivo**: si può modificare anche solo una porzione informativa mantenendo invariati gli altri attributi.

Tutto ciò ha reso possibile l'incremento progressivo delle integrazioni in Teseo in quanto ogni nuova sorgente può popolare solo gli aspetti di sua competenza senza forzare migrazioni globali del modello.

Un elemento rilevante emerso dall'analisi della documentazione è il ruolo dei **key aspects**: questi sono aspetti speciali che identificano in modo univoco un'entità e da cui viene derivata la relativa URN. Questo meccanismo rende coerente il *primary-key lookup* nel grafo dei metadati e assicura la stabilità degli identificativi tra ingestion successive. In particolare, il formato della URN viene costruito in modo deterministico a partire dai campi del key aspect, evitando quindi collisioni semantiche tra entità simili ma appartenenti a piattaforme o ambienti diversi [14].

2.4.2 Entity Registry e governabilità del modello

Un altro aspetto fondamentale per la sostenibilità architetturale è la modalità con cui DataHub definisce il proprio schema globale: il metadata model è orchestrato tramite **Entity Registry** (`entity-registry.yml`), il quale si occupa della dichiarazione delle entità e dei relativi aspetti associati. L'Entity Registry viene validato in fase di avvio del *Metadata Service*.

Questo sistema di tipo **dichiarativo** ha due implicazioni pratiche rilevanti:

- l'estensione del modello non richiede interventi distruttivi sul core, ma aggiunte controllate di entità/aspetti dichiarati;
- la coerenza tra configurazione e schema effettivo viene verificata in modo centralizzato, riducendo pertanto errori di modellazione in fase di evoluzione.

Dal punto di vista progettuale, l'Entity Registry rappresenta quindi il *Model Manager*, il cui compito è bilanciare estensibilità e controllo semantico [14]. La Figura 2.3 sintetizza il modello in forma semplificata: un'entità (ad esempio un Dataset) è identificata in modo univoco da una URN derivata dal proprio key aspect ed è descritta da aspetti indipendenti (schema, ownership, tag, glossary terms, lineage), aggiornabili separatamente.

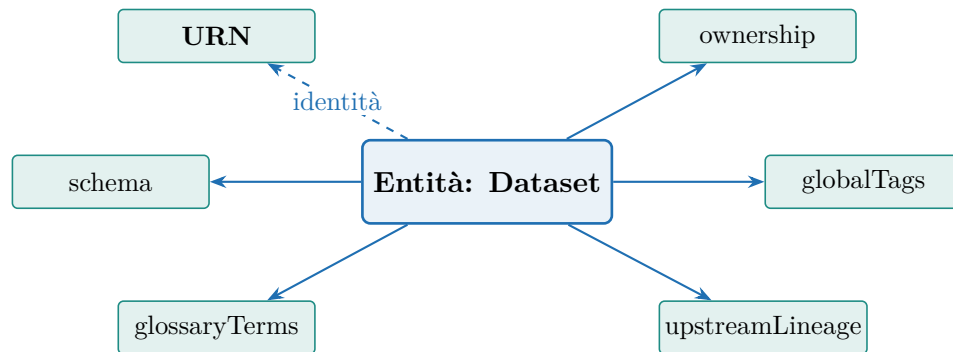


Figura 2.3: Metadata Model semplificato

2.4.3 Relazioni e lettura del lineage

Avere un catalogo ordinato e completo dei dati è molto importante, ma, senza poter visualizzare le relazioni tra questi dati e le loro dipendenze, non c'è un vero valore aggiunto: **è come avere una cassaforte piena d'oro, ma senza avere la combinazione per aprirla.**

Il vero potenziale di Teseo emerge nella capacità di esplicitare le relazioni tra le entità: in particolare, la rappresentazione upstream/downstream abilita la lettura delle dipendenze tra sorgenti (ad esempio Snowflake), trasformazioni (ad esempio dbt) e layer di consumo analitico (ad esempio Power BI). Infatti, come già accennato in precedenza, questo supporta impact analysis più affidabili e riduce la dipendenza da conoscenza tacita distribuita nel team (favorendo quindi una migliore comprensione ai non esperti e ai nuovi membri del team).

L'adozione di un modello a grafo consente inoltre di integrare dimensioni diverse nello stesso percorso di lineage: non solo legami tecnici tra tabelle e job, ma anche ownership, domini e documentazione, creando una vista unificata utile sia a profili tecnici sia a stakeholder di data governance.

2.4.4 Versionamento e coerenza temporale

Il versionamento degli aspetti del metadata è rilevante per analizzarne le evoluzioni nel tempo (ad esempio aggiornamenti di schema o cambiamenti nelle sue proprietà): questo sistema abilita confronti storici e verifica dell'evoluzione del catalogo senza perdere lo snapshot corrente.

Occorre però distinguere **versioned aspects** e **timeseries aspects** a livello operativo:

- **versioned aspects**: rappresentano lo stato più recente di un aspetto, con la possibilità di accedere a versioni precedenti per scopi di audit o rollback;
- **timeseries aspects**: rappresentano eventi ordinati nel tempo (ad esempio profiling e usage statistics) e sono interrogabili per intervallo temporale.

Questa separazione consente di ottimizzare persistenza e query in base al tipo di informazione ricercata [14]. Per Teseo, tale capacità è un elemento chiave nel mantenere auditabilità tecnica durante le fasi di estensione delle pipeline di ingestion, soprattutto nei casi in cui la verifica richiede sia lo stato corrente dell'asset sia le sue versioni precedenti.

Qui sotto un esempio di ingestion di un timeseries aspect:

```

1 from datahub.metadata.schema_classes import (
2     ChangeTypeClass,
3     DatasetFieldUsageCountsClass,
4     DatasetUsageStatisticsClass,
5 )
6 from datahub.emitter.kafka_emitter import DatahubKafkaEmitter
7 from datahub.emitter.mcp import MetadataChangeProposalWrapper
8 from datahub.emitter.rest_emitter import DatahubRestEmitter
9
10 usageStats = DatasetUsageStatisticsClass(
11     timestampMillis=1629840771000,
12     uniqueUserCount=10,
13     totalSqlQueries=20,
14     fieldCounts=[
15         DatasetFieldUsageCountsClass(
16             fieldPath="col1",
17             count=10
18         )
19     ]
20 )
21
22 mcpw = MetadataChangeProposalWrapper(
23     entityType="dataset",
24     aspectName="datasetUsageStatistics",

```

```

25     changeType=ChangeTypeClass.UPSERT,
26     entityUrn="urn:li:dataset:(urn:li:dataPlatform:hive,
SampleHiveDataset,PROD)",
27     aspect=usageStats,
28 )
29
30 # Instantiate appropriate emitter (kafka_emitter/rest_emitter)
31 # my_emitter = DatahubKafkaEmitter(""<config>"")
32 my_emitter = DatahubRestEmitter("http://localhost:8080")
33 my_emitter.emit(mcpw)

```

Listing 2.1: Esempio di ingestion di un timeseries aspect

2.4.5 Ciclo degli eventi metadata (MCP e MCL)

Un altro elemento che è stato rilevante nella scelta di DataHub Core è il modello degli **Eventi Metadata**: le modifiche ai metadati vengono proposte come **Metadata Change Proposal (MCP)** e, una volta applicate al grafo, producono **Metadata Change Log (MCL)**. In pratica, MCP rappresenta la richiesta di mutazione su un aspetto specifico, mentre MCL rappresenta il cambiamento effettivamente committato e osservabile a valle [15].

Dal punto di vista architetturale, questa distinzione chiarisce il percorso di **affidabilità** verso il catalogo dei metadati:

- il Metadata Change Proposal abilita l'ingresso controllato della proposta di modifica (CREATE, UPSERT, DELETE) su un aspetto;
- il Metadata Change Log traccia la modifica applicata, includendo anche stato precedente e metadati di sistema funzionali all'audit.

Questo sistema è molto utile perché collega in modo formale la configurazione di ingestion al comportamento osservato nel catalogo, fornendo un perimetro più chiaro per troubleshooting e validazione post-run [15].

2.5 Ingestion layer multi-source

L'ingestion layer costituisce il punto di contatto tra sorgente e metadata platform. In DataHub, questo livello è supportato da un Ingestion Framework modulare che permette di configurare le ingestion tramite ricette YAML, con supporto a modalità di integrazione differenti [5, 12].

Come evidenziato anche dalla documentazione architetturale ufficiale, il perno dell'ingestion è il **Metadata Change Proposal (MCP)** sopra citato: ogni cambiamento viene modellato come proposta di modifica di un aspetto del grafo. Questo consente di uniformare canali di ingresso eterogenei verso un formato semantico comune, mantenendo coerenza tra sorgenti e modello interno [15, 16]. La Figura 2.4 riassume questo percorso: le sorgenti vengono lette dall'Ingestion Framework configurato tramite recipe YAML, che genera i Metadata Change Proposal recapitati al Metadata Service in modalità asincrona (Kafka) o sincrona (HTTP) e infine persistiti e indicizzati.

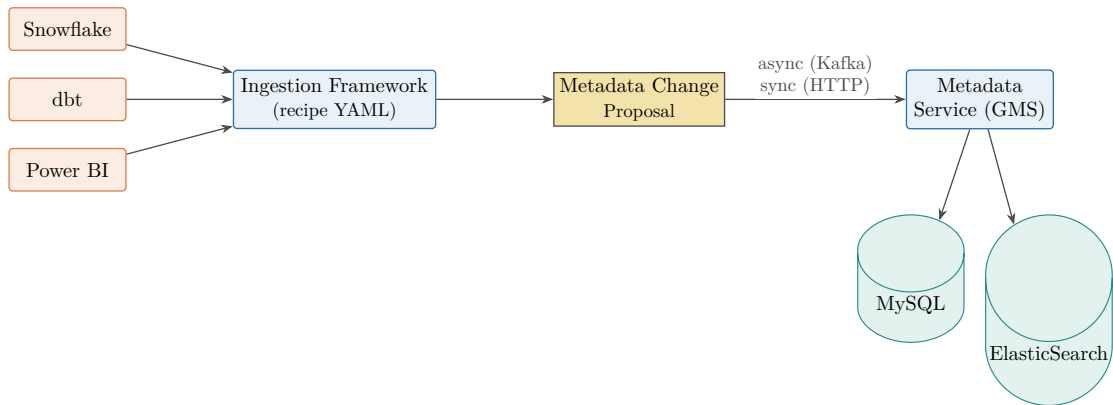


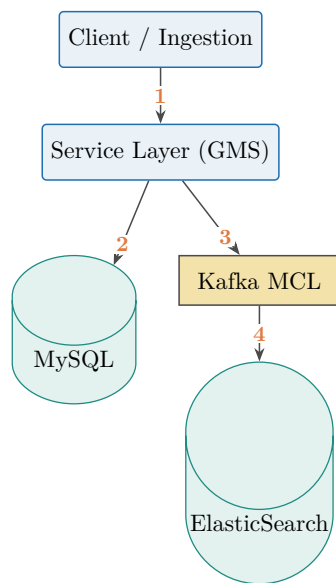
Figura 2.4: Flusso di metadata ingestion

2.5.1 Metodi di integrazione supportati

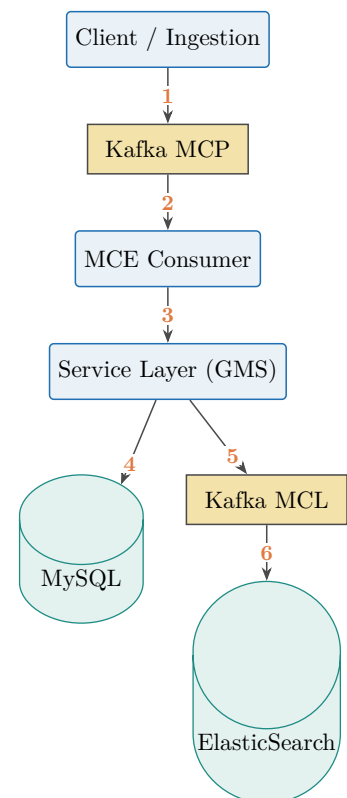
DataHub mette a disposizione tre modalità principali per *ingestionare* i metadati: configurazione da interfaccia (*UI*), esecuzione da *CLI* (mediante ricette YAML; la scelta nel mio caso è ricaduta su questo perché è dichiarativo e riproducibile) e da API (*SDK*). La coesistenza di queste opzioni è utile perché consente di scegliere il canale più adatto al caso d'uso (un'opzione non preclude le altre).

Un punto rilevante, non esplicitato in precedenza, è che un Metadata Change Proposal può essere pubblicato sia in modalità **asincrona** (topic Kafka) sia in modalità **sincrona** (endpoint HTTP del service tier). Questa doppia modalità permette di bilanciare scalabilità e controllo transazionale in base al tipo di integrazione: pipeline ad alto volume possono privilegiare Kafka, mentre processi che richiedono feedback immediato possono usare il canale HTTP [16]. La Figura 2.5 confronta le due modalità: nella pubblicazione sincrona il client scrive direttamente sul Service Layer (pochi passaggi, feedback immediato), mentre in quella asincrona la proposta transita prima da un topic Kafka e da un consumer dedicato, introducendo più passaggi a fronte di un maggiore decoupling e scalabilità.

Pubblicazione sincrona (HTTP)



Pubblicazione asincrona (Kafka)

**Figura 2.5:** Pubblicazione sincrona e asincrona di un MCP

Le integrazioni possono seguire approccio *pull-based* (raccolta periodica dalla sorgente) o *push-based* (emissione di eventi metadata dal sistema sorgente). Nel perimetro realizzato, la strategia principale è stata quella pull-based per le sorgenti prioritarie (nel mio caso Snowflake principalmente) poiché più controllabile nella fase iniziale e più semplice da standardizzare nei cicli operativi.

Come visto dall'architettura, in scenari pull-based, le pipeline possono essere orchestrate anche da scheduler esterni (ad esempio Airflow o Spark), mentre negli scenari push-based la pubblicazione degli MCP può avvenire direttamente tramite emitter dedicati [16].

2.5.2 Pattern implementativo in Teseo

Il pattern adottato per ogni nuova sorgente è stato strutturato in step ripetibili:

1. definizione del perimetro informativo da estrarre (asset, schema, usage, lineage, ownership...);
2. scrittura della recipe YAML con parametri di connessione, opzioni funzionali e configurazione di source/sink;
3. esecuzione controllata della ingestion in ambiente locale da CLI;
4. verifica in DataHub della consistenza tra entità ingestite, relazioni e ricerca;
5. consolidamento delle informazioni operative per esecuzioni successive.

Questo approccio ha permesso di ridurre la difficoltà nella gestione delle nuove integrazioni e di mantenere prevedibile il comportamento di Teseo durante l'espansione del catalogo.

Come citato nella sezione precedente, i Metadata Change Proposal vengono poi processati da componenti applicativi dedicati (*consumer job*) che li applicano al Metadata Service. Questo passaggio è fondamentale perché separa il momento di emissione dalla fase di persistenza della modifica, rendendo più robusto l'intero ciclo di ingestion in presenza di sorgenti eterogenee [16].

2.6 Exposure Layer: UI, GraphQL e API

L'accesso ai metadata in Teseo è stato progettato secondo il principio "*multiple interfaces, single source of truth*": canali diversi per utenti e automazioni, ma un unico grafo di metadata come base informativa consistente.

2.6.1 UI per navigazione

La UI React di DataHub è stata utilizzata come canale principale per attività quotidiane di discovery, consultazione schemi, verifica ownership e navigazione del lineage. Questo riduce la necessità di strumenti custom per use case standard e accelera l'adozione della piattaforma da parte del team.

2.6.2 GraphQL per interrogazione

GraphQL fornisce un'API tipizzata e orientata alle entità, coerente con il modo in cui la UI interagisce con il catalogo [17]. In Teseo, questa caratteristica è stata considerata strategica per due motivi: semplicità nell'esprimere query di navigazione e allineamento naturale con casi d'uso esplorativi dalla UI.

2.6.3 SDK e API per automazione

Python/Java SDK rappresentano l'opzione raccomandata per operazioni programmatiche avanzate, lasciando a GraphQL un ruolo prevalentemente orientato a scenari *user-facing* [13]. La Tabella 2.1 confronta le principali interfacce e API esposte da Teseo, evidenziandone vantaggi e limiti rispetto ai diversi casi d'uso (interazione umana o automazione).

API	Definizione	Vantaggi	Limiti
Python SDK	SDK	Molto flessibile e bulk-oriented	Richiede comprensione dei metadata change event
Java SDK	SDK	Molto flessibile e bulk-oriented	Richiede comprensione dei metadata change event
GraphQL API	GraphQL	Intuitiva e UI-oriented	Meno flessibile e richiede conoscenza di GraphQL
OpenAPI	Low-level	Massima potenza e flessibilità	Complessa e senza SDK dedicato

Tabella 2.1: Confronto tra interfacce e API di DataHub

2.7 Sicurezza e osservabilità

La **sicurezza** è stata trattata come requisito architetturale trasversale (infatti, come detto in precedenza, l'ambiente DataHub è stato configurato con autenticazione

attiva per evitare accessi anonimi al metadata service e consentire una gestione controllata dei token di integrazione).

Le principali linee adottate sono:

- separazione tra credenziali di Teseo/DataHub e credenziali della sorgente ingestionata;
- limitazione dei privilegi alle sole operazioni necessarie per l'ingestion (*least privilege*);
- tracciabilità delle operazioni tramite log e run identificabili;
- separazione dei ruoli tra amministrazione piattaforma, gestione integrazioni e consultazione.

Manage Access Tokens

Manage Access Tokens for use with DataHub APIs.

Personal Access Tokens

Personal Access Tokens allow you to make programmatic requests to DataHub's APIs. They inherit your privileges and have a finite lifespan. Do not share Personal Access Tokens.

+ Generate new token

Name	Description	Expires At
------	-------------	------------



No Access Tokens!

Figura 2.6: Sezione di Teseo per la gestione dei token

La Figura 2.6 mostra la sezione dell'interfaccia di Teseo dedicata alla generazione e gestione dei *personal access token*, utilizzati per autenticare le integrazioni esterne verso il metadata service mantenendo separazione e revocabilità delle credenziali. Dal punto di vista operativo, la combinazione di logging applicativo, controllo dei run di ingestion e validazione post-run in UI costituisce il meccanismo principale di osservabilità del sistema.

2.8 Lifecycle del metadato nell'architettura Teseo

Per comprendere pienamente il disegno architetturale, bisogna vedere Teseo come un sistema governato dal lifecycle continuo del metadato, più che come un insieme statico di componenti. Infatti, ogni informazione attraversa fasi distinte: acquisizione, normalizzazione, persistenza, indicizzazione, consultazione e aggiornamento.

Nella fase di **acquisizione**, i connettori prelevano metadati tecnici dalle sorgenti (strutture, dipendenze, attributi operativi...), riducendo al minimo trasformazioni non deterministiche. La qualità di questa fase è critica: errori di mapping iniziale tendono a propagarsi a valle e a compromettere la leggibilità complessiva del catalogo.

La fase successiva è la **normalizzazione**, nella quale i dati estratti vengono mappati sul metadata model. In Teseo, questo passaggio è stato trattato come punto di controllo architetturale: convenzioni di naming, coerenza degli identificativi e granularità delle entità sono state mantenute uniformi tra sorgenti diverse per evitare duplicazioni semantiche.

Segue poi la fase di **persistenza**, in cui le informazioni normalizzate vengono registrate come aspetti di entità identificabili in modo univoco. In questa fase il principio cardine è l'idempotenza: rieseguire un'ingestion non deve produrre divergenze strutturali, ma aggiornare lo stato in modo prevedibile.

L'**indicizzazione** rappresenta la fase che rende i metadati fruibili su larga scala: ricerca, filtri, navigazione e accesso ai nodi più rilevanti dipendono dalla qualità degli indici.

Nella fase di **consultazione**, utenti tecnici e stakeholder interrogano il catalogo per use case differenti: data discovery, impact analysis, verifica ownership e lineage navigation. L'architettura deve quindi garantire tempi di risposta adeguati e una rappresentazione comprensibile delle dipendenze.

Infine, la fase di **aggiornamento** continuo chiude e riapre il ciclo: cambiamenti nelle sorgenti richiedono refresh periodici o event-driven del catalogo, preservando la coerenza tra stato reale dei sistemi e stato rappresentato in piattaforma.

2.9 Scenari di rischio e strategie di mitigazione

Durante la progettazione sono stati identificati alcuni rischi tipici delle metadata platform centralizzate. La mitigazione di tali rischi è stata incorporata nel disegno architetturale.

2.9.1 Rischio di incompletezza del lineage

Il primo rischio è ottenere un catalogo apparentemente ricco ma con lineage incompleto sui tratti più critici: ciò accade quando alcune sorgenti forniscono metadati limitati o quando i permessi disponibili non consentono estrazione di alcune relazioni avanzate.

La mitigazione adottata è stata duplice, ovvero esplicitare il livello di copertura per ogni integrazione e separare chiaramente metadati "core" da metadati "estesi": in questo modo, la piattaforma mantiene una configurazione di ingestion affidabile (*core*), rendendo visibili le aree in cui la copertura è parziale.

2.9.2 Rischio di deriva semantica

Con l'aumento del numero di sorgenti cresce la probabilità di incongruenze semantiche (nomi discordanti, descrizioni divergenti, ownership duplicata), aumentando la complessità di gestione e riducendo l'efficacia del catalogo, seppur in presenza di ingestion tecnicamente corretta.

Per ridurre questo problema, si è deciso di usare convenzioni comuni e controlli periodici sulla qualità dei metadati: infatti la normalizzazione non è stata trattata come *una tantum*, ma come pratica continua.

2.9.3 Rischio di regressione

In ambienti on-prem, l'aggiornamento dei componenti può introdurre regressioni funzionali o incompatibilità indirette: per questo motivo, gli aggiornamenti sono stati trattati come cambiamenti architetturali da validare su percorso controllato, includendo verifica di ingestion, ricerca e lineage, prima del consolidamento.

2.10 Benefici architetturali osservati

La validazione condotta nel corso del progetto ha evidenziato benefici concreti direttamente riconducibili alle scelte architetturali.

- **Riduzione del tempo di orientamento:** l'accesso centralizzato ai metadati riduce il tempo necessario per comprendere origine e uso degli asset;
- **Maggiore prevedibilità delle modifiche:** la disponibilità del lineage migliora la capacità di stimare impatti prima di interventi su sorgenti o trasformazioni;
- **Migliore collaborazione tra ruoli:** data engineer, analyst e figure di governance condividono una base informativa comune;
- **Scalabilità organizzativa:** l'onboarding di nuove sorgenti segue pattern già consolidati, con minore dipendenza dalla conoscenza individuale.

In sintesi, il valore dell'architettura non è limitato alla correttezza tecnologica, ma si riflette nella capacità dell'organizzazione di prendere decisioni più rapide e più affidabili in presenza di ecosistemi dati complessi.

2.11 Posizionamento dei moduli evolutivi

Pur mantenendo il focus del capitolo sull'architettura core, è utile chiarire il posizionamento dei moduli evolutivi già previsti nel disegno complessivo. In particolare, le funzionalità avanzate di governance e le integrazioni AI tramite protocollo MCP sono state trattate come layer sovrastanti rispetto al nucleo DataHub, non come sostituti del catalogo metadata (sono degli strumenti aggiuntivi, non dei componenti centrali).

Questa scelta preserva un principio fondamentale: il catalogo resta la fonte autorevole dei metadata, mentre i moduli evolutivi agiscono come canali di fruizione, automazione o supporto decisionale.

Capitolo 3

Ingestion multi-source e lineage end-to-end

Questo capitolo descrive l'implementazione dei flussi di ingestion multi-source realizzati nel progetto Teseo, con focus su tre integrazioni principali: **Snowflake**, **dbt** e **Power BI**. L'obiettivo tecnico è costruire un grafo dei metadati capace di rappresentare la dipendenza completa tra sorgenti, trasformazioni e livello di consumo analitico, fino al **column-level lineage** end-to-end.

L'approccio adottato è stato progressivo e ripetibile: prima consolidamento del dominio dati su Snowflake con DDL strutturati per layer, poi attivazione del lineage trasformativo tramite dbt, in seguito pubblicazione del livello di consumo con un semantic model Power BI locale *mockato* ed infine integrazione con un ambiente di produzione Power BI. Il risultato è una catena end-to-end osservabile nel catalogo metadati.

3.1 Snowflake: DDL e setup

Snowflake è stato usato come piattaforma dati cloud per consolidare i layer del dominio e rendere esplicite le dipendenze fisiche tra oggetti. La separazione in database e schemi (RL/QL/BL + metadati framework) ha fornito un perimetro chiaro per ingestion e governance tecnica [3].

Nel progetto, la struttura è stata organizzata su quattro database principali:

- DB_DEMO_RL (Raw Layer, ovvero il livello in cui i dati vengono caricati in forma grezza, senza trasformazioni o pulizie significative);
- DB_DEMO_QL (Quality Layer, ovvero il livello in cui vengono applicate regole di qualità, pulizia e validazione sui dati provenienti dal Raw Layer);

- DB_DEMO_BL (Business Layer, ovvero il livello in cui vengono applicate logiche di business e aggregazioni sui dati precedentemente puliti e validati);
- DB_DEMO_FMK (Framework Metadata, ovvero il layer dei metadati e delle procedure di framework, il cui scopo è quello di fornire una struttura coerente e organizzata per i successivi passi di ingestion).

3.1.1 Setup iniziale e validazione permessi

Prima del deployment completo operativo ho usato degli script SQL di bootstrap e verifica dei permessi, in particolare:

```

1 show grants to user MDOMIZIO;
2 show grants to role TESEO_INTEGRATION_ROLE;
3 show grants on warehouse COMPUTE_WH;
4 show grants on database SNOWFLAKE;
```

Listing 3.1: Verifica grants iniziali

```

1 use role TESEO_INTEGRATION_ROLE;
2 use warehouse COMPUTE_WH;
3
4 create database if not exists DB_DEMO_RL;
5 create database if not exists DB_DEMO_QL;
6 create database if not exists DB_DEMO_BL;
7 create database if not exists DB_DEMO_FMK;
8
9 show databases like 'DB_DEMO_%';
10
11 create schema if not exists DB_DEMO_RL.RL_TMP_PROBE;
12 create schema if not exists DB_DEMO_QL.QL_TMP_PROBE;
13 create schema if not exists DB_DEMO_BL.BL_TMP_PROBE;
14 create schema if not exists DB_DEMO_FMK.FMK_TMP_PROBE;
```

Listing 3.2: Bootstrap database e schema

```

1 show grants to user MDOMIZIO;
2 show grants to role TESEO_INTEGRATION_ROLE;
3 use role TESEO_INTEGRATION_ROLE;
4 use warehouse COMPUTE_WH;
5
6 show databases like 'DB_DEMO_%';
7 show schemas in database DB_DEMO_RL;
8 show schemas in database DB_DEMO_QL;
9 show schemas in database DB_DEMO_BL;
10 show schemas in database DB_DEMO_FMK;
11
12 select count(*) as account_usage_access_ok
```

```

13 from SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
14 where start_time >= dateadd(day, -1, current_timestamp());
15
16 select count(*) as info_schema_access_ok
17 from DB_DEMO_QL.INFORMATION_SCHEMA.COLUMNS;
18
19 select count(*) as process_table_rows
20 from DB_DEMO_FMK.META.MDT_PROCESS_TABLE;
21
22 create schema if not exists DB_DEMO_RL.RL_TMP_TEST;
23 create or replace table DB_DEMO_RL.RL_TMP_TEST.TB_SMOKE (ID number
    , TS timestamp_ntz);
24 insert into DB_DEMO_RL.RL_TMP_TEST.TB_SMOKE values (1,
    current_timestamp());
25 select * from DB_DEMO_RL.RL_TMP_TEST.TB_SMOKE;
26 drop table if exists DB_DEMO_RL.RL_TMP_TEST.TB_SMOKE;

```

Listing 3.3: Validazione permessi e smoke test runtime

Le verifiche includono la visibilità degli oggetti, il contesto runtime (utente, ruolo e warehouse) e l'accesso a `SNOWFLAKE.ACCOUNT_USAGE`, necessario per lineage e usage avanzati lato connettore.

3.1.2 Deployment DDL del dominio e delle procedure

Dopo aver effettuato il bootstrap e le validazioni iniziali, il modello dati è stato creato con un pacchetto DDL (Data Definition Language) suddiviso in cinque file tematici:

- `DDL-01_FOUNDATION.sql`: setup database, schema e tabelle metadata framework;
- `DDL-02_MDM_LAYERS.sql`: dimensioni MDM (Master Data Management, ovvero gestione dei dati anagrafici e master, per esempio store, product, loyalty card...) su RL, QL e BL;
- `DDL-03_TDM_FACT_LAYERS.sql`: fatti TDM (Transactional Data Management, ovvero gestione dei dati transazionali, per esempio receipt line, footfall, KPI aggregati...) su RL, QL e BL;
- `DDL-04_DATAMART_VIEWS.sql`: viste aggregate tra cui `VW_DM_TDM_TIMEBAND` e `VW_DM_TDM_DAILY`;
- `DDL-05_PROCEDURES.sql`: stored procedures di caricamento `SP_RL_LOADING`, `SP_QL_LOADING`, `SP_BL_LOADING`.

Questa struttura rende osservabile un lineage multilivello (stage esterno, raw, quality, business, datamart) e fornisce una base fisica coerente per l'ingestion in DataHub.

Dal punto di vista strutturale, il pattern applicato alle entità core (come TB_RECEIPT_LINE, TB_STORE e TB_PRODUCT) è coerente su tutti i layer:

- **RL**: tabella raw con naming legacy e tipizzazione permissiva (VARCHAR estesi), al fine di massimizzare la copertura dei dati sorgente;
- **QL**: stream su RL, vista di normalizzazione (rename + quality rules), tabella tipizzata con vincoli primary key e foreign key, e tabella discard;
- **BL**: stream su QL, vista di arricchimento business, tabella consolidata e tabella discard.

Per evitare una descrizione troppo astratta, di seguito riporto alcuni estratti DDL rappresentativi di varie operazioni svolte (fondazione, raw, typed/quality, business enrichment, datamart e procedura framework).

```

1 CREATE DATABASE IF NOT EXISTS DB_DEMO_RL;
2 CREATE DATABASE IF NOT EXISTS DB_DEMO_QL;
3 CREATE DATABASE IF NOT EXISTS DB_DEMO_BL;
4 CREATE DATABASE IF NOT EXISTS DB_DEMO_FMK;
5
6 CREATE SCHEMA IF NOT EXISTS DB_DEMO_RL.RL_TDM;
7 CREATE SCHEMA IF NOT EXISTS DB_DEMO_RL.RL_MDM;
8 CREATE SCHEMA IF NOT EXISTS DB_DEMO_QL.QL_TDM;
9 CREATE SCHEMA IF NOT EXISTS DB_DEMO_QL.QL_TDM_DSC;
10 CREATE SCHEMA IF NOT EXISTS DB_DEMO_QL.QL_MDM;
11 CREATE SCHEMA IF NOT EXISTS DB_DEMO_QL.QL_MDM_DSC;
12 CREATE SCHEMA IF NOT EXISTS DB_DEMO_BL.BL_TDM;
13 CREATE SCHEMA IF NOT EXISTS DB_DEMO_BL.BL_TDM_DSC;
14 CREATE SCHEMA IF NOT EXISTS DB_DEMO_BL.BL_MDM;
15 CREATE SCHEMA IF NOT EXISTS DB_DEMO_BL.BL_MDM_DSC;
```

Listing 3.4: Fondazione tecnica (database e schemi)

Flusso completo di esempio su TB_RECEIPT_LINE:

```

1 CREATE OR REPLACE TABLE DB_DEMO_RL.RL_TDM.TB_RECEIPT_LINE (
2   TS_INGESTION TIMESTAMP_NTZ(9),
3   DTVN VARCHAR, NEGO VARCHAR, NSCN VARCHAR, NRIG VARCHAR,
4   BARC VARCHAR, PZZI VARCHAR, PRZU VARCHAR, ISCN VARCHAR, INTT
5     VARCHAR,
6   PIVA VARCHAR, ORVN VARCHAR, NCAS VARCHAR, COPR VARCHAR,
7   FRES VARCHAR, FPRM VARCHAR, CPRM VARCHAR, MPAG VARCHAR, CFID
8     VARCHAR);
```

```

7  DIVI VARCHAR, CAMB VARCHAR, STAG VARCHAR, COLL VARCHAR,
8  MARC VARCHAR, REPA VARCHAR, CATG VARCHAR, DCAR VARCHAR
9  );

```

Listing 3.5: Raw layer (struttura sorgente legacy)

```

1  CREATE OR REPLACE STREAM DB_DEMO_QL.QL_TDM.ST_QL_TB_RECEIPT_LINE
2  ON TABLE DB_DEMO_RL.RL_TDM.TB_RECEIPT_LINE;
3
4  CREATE OR REPLACE VIEW DB_DEMO_QL.QL_TDM.TVW_TB_RECEIPT_LINE AS
5  SELECT
6    DTVN AS SALE_DATE,
7    NEGO AS STORE_CODE,
8    NSCN AS RECEIPT_NUM,
9    NRIG AS LINE_NUM,
10   BARC AS ITEM_BARCODE,
11   LEFT(ORVN, 2) AS SALE_HOUR,
12   CASE WHEN UPPER(FRES) IN ('Y','S','1') THEN 'Y' ELSE 'N' END AS
13     FL_RETURN,
14   CASE WHEN UPPER(FPRM) IN ('Y','S','1') THEN 'Y' ELSE 'N' END AS
15     FL_PROMO,
16   CASE
17     WHEN REPLACE(CFID, ' ', '') IN ('O','OO','NA','NULL','') THEN
18       NULL
19     ELSE REPLACE(CFID, ' ', '')
20   END AS LOYALTY_CARD_ID
21 FROM DB_DEMO_QL.QL_TDM.ST_QL_TB_RECEIPT_LINE
22 WHERE METADATA\$_ACTION = 'INSERT';
23
24 CREATE OR REPLACE TABLE DB_DEMO_QL.QL_TDM.TB_RECEIPT_LINE (
25   SALE_DATE DATE NOT NULL,
26   STORE_CODE VARCHAR(10) NOT NULL,
27   RECEIPT_NUM VARCHAR(20) NOT NULL,
28   LINE_NUM NUMBER(6,0) NOT NULL,
29   ITEM_BARCODE VARCHAR(14),
30   NET_AMOUNT NUMBER(12,2),
31   CONSTRAINT TB_RECEIPT_LINE_PK PRIMARY KEY (SALE_DATE, STORE_CODE
32     , RECEIPT_NUM, LINE_NUM)
33 );

```

Listing 3.6: Quality layer (stream + normalizzazione + tabella tipizzata)

```

1  CREATE OR REPLACE STREAM DB_DEMO_BL.BL_TDM.ST_BL_TB_RECEIPT_LINE
2  ON TABLE DB_DEMO_QL.QL_TDM.TB_RECEIPT_LINE;
3
4  CREATE OR REPLACE VIEW DB_DEMO_BL.BL_TDM.TVW_TB_RECEIPT_LINE AS
5  SELECT

```



```

6  CURRENT_TIMESTAMP() AS TS_INGESTION,
7  R.SALE_DATE,
8  R.STORE_CODE,
9  R.RECEIPT_NUM,
10 R.LINE_NUM,
11 ROUND(R.NET_AMOUNT * COALESCE(R.EXCHANGE_RATE, 1), 2) AS
    NET_AMOUNT_EUR,
12 CASE
13     WHEN R.SALE_HOUR BETWEEN 0 AND 11 THEN 'MORNING'
14     WHEN R.SALE_HOUR BETWEEN 12 AND 17 THEN 'AFTERNOON'
15     WHEN R.SALE_HOUR BETWEEN 18 AND 23 THEN 'EVENING'
16     ELSE 'UNKNOWN'
17 END AS TIME_BAND,
18 L.CUSTOMER_ID
19 FROM DB_DEMO_BL.BL_TDM.ST_BL_TB_RECEIPT_LINE R
20 LEFT JOIN DB_DEMO_BL.BL_MDM.TB_LOYALTY_CARD L
21     ON R.LOYALTY_CARD_ID = L.LOYALTY_CARD_ID
22 WHERE METADATA\${ACTION} = 'INSERT';

```

Listing 3.7: Business layer (arricchimento e consolidamento)

```

1  CREATE OR REPLACE VIEW DB_DEMO_BL.BL_TDM.VW_DM_TDM_TIMEBAND AS
2  SELECT
3      R.SALE_DATE,
4      R.STORE_CODE,
5      R.TIME_BAND,
6      COUNT(DISTINCT R.RECEIPT_NUM) AS NUM_RECEIPTS,
7      SUM(R.NET_AMOUNT_EUR) AS NET_REVENUE_EUR,
8      F.TOTAL_ENTRIES,
9      CASE
10         WHEN F.TOTAL_ENTRIES > 0
11         THEN ROUND(COUNT(DISTINCT R.RECEIPT_NUM)::FLOAT / F.
            TOTAL_ENTRIES * 100, 2)
12         ELSE NULL
13     END AS CONVERSION_RATE
14 FROM DB_DEMO_BL.BL_TDM.TB_RECEIPT_LINE R
15 LEFT JOIN (
16     SELECT EVENT_DATE, STORE_CODE, TIME_BAND,
17            SUM(ENTRIES_COUNT) AS TOTAL_ENTRIES
18     FROM DB_DEMO_BL.BL_TDM.TB_FOOTFALL
19     GROUP BY EVENT_DATE, STORE_CODE, TIME_BAND
20 ) F
21     ON R.SALE_DATE = F.EVENT_DATE
22     AND R.STORE_CODE = F.STORE_CODE
23     AND R.TIME_BAND = F.TIME_BAND
24 GROUP BY R.SALE_DATE, R.STORE_CODE, R.TIME_BAND, F.TOTAL_ENTRIES;

```

Listing 3.8: Datamart (aggregazione su vista business)

Flusso logico osservabile in ordine:

1. RL_TDM.TB_RECEIPT_LINE
2. QL_TDM.TVW_TB_RECEIPT_LINE / TB_RECEIPT_LINE
3. BL_TDM.TVW_TB_RECEIPT_LINE / TB_RECEIPT_LINE
4. BL_TDM.VW_DM_TDM_TIMEBAND / VW_DM_TDM_DAILY

Infine, come ultimo passo, il framework automatizza il caricamento dei layer con stored procedures dedicate. Nel caso QL (quello usato nel flusso completo su TB_RECEIPT_LINE) la procedura non si limita a chiamare una vista, ma legge la configurazione dal framework metadata e decide dinamicamente il metodo di caricamento:

```

1 CREATE OR REPLACE PROCEDURE DB_DEMO_FMK.QL_FMK.SP_QL_LOADING (
2   "TABLE_FULL_NAME" VARCHAR,
3   "CALLER"           VARCHAR DEFAULT 'manual'
4 )
5 RETURNS VARCHAR
6 LANGUAGE PYTHON
7 RUNTIME_VERSION = '3.11'
8 PACKAGES = ('snowflake-snowpark-python')
9 HANDLER = 'main'
10 EXECUTE AS CALLER
11 AS '
12
13 PROCESS_TABLE = "DB_DEMO_FMK.META.MDT_PROCESS_TABLE"
14
15 def get_process_method(session, table_full_name):
16     db, sch, tbl = table_full_name.split(".")
17     proc = session.table(PROCESS_TABLE)
18     row = proc.filter(
19         (col("GN_TARGET_TABLE_DATABASE_NAME") == db) &
20         (col("GN_TARGET_TABLE_SCHEMA_NAME") == sch) &
21         (col("GN_TARGET_TABLE_NAME") == tbl)
22     ).select("GN_PROCESS_METHOD").collect()
23     return row[0][0] if row else "APPEND"
24
25 def load_data(session, table_full_name, source_tbl, pk_names,
26               method):
27     target_cols = session.table(table_full_name).columns
28     cols_str = ", ".join(target_cols)
29
30     if method == "APPEND":
31         session.sql(f"INSERT INTO {table_full_name} ({cols_str})
32         SELECT {cols_str} FROM {source_tbl}").collect()
33     elif method == "REPLACE":

```

```

32     session.sql(f"TRUNCATE TABLE {table_full_name}").collect()
33     session.sql(f"INSERT INTO {table_full_name} ({cols_str})
    SELECT {cols_str} FROM {source_tbl}").collect()
34 elif method == "MERGE" and pk_names:
35     on_clause = " AND ".join([f"T.{k} = S.{k}" for k in pk_names])
36     merge_sql = f"""
37         MERGE INTO {table_full_name} T
38         USING {source_tbl} S ON {on_clause}
39         WHEN MATCHED THEN UPDATE SET ...
40         WHEN NOT MATCHED THEN INSERT (...)
41     """
42     session.sql(merge_sql).collect()
43
44 # ... body python ...
45 ';
```

Listing 3.9: Sintassi stored procedure e strategie di load

Il main della stessa procedura mostra in modo esplicito il flusso tecnico completo del controllo qualità prima dell'effettivo load finale:

```

1 def main(session, table_full_name, caller):
2     db, sch, tbl = table_full_name.split(".")
3     dsc_tbl = f"{db}.{sch}_DSC.DSC_{tbl}"
4     tvw = f"{db}.{sch}.TVW_{tbl}"
5     batch_tbl = f"{db}.{sch}.BATCH_{tbl}"
6     cast_tbl = f"{batch_tbl}_CAST"
7
8     session.sql(f"CREATE OR REPLACE TEMPORARY TABLE {batch_tbl} LIKE
    {tvw}").collect()
9     session.sql(f"CREATE OR REPLACE TEMPORARY TABLE {cast_tbl} LIKE
    {table_full_name}").collect()
10    session.sql(f"INSERT INTO {batch_tbl} SELECT * FROM {tvw}").
    collect()
11
12    pk_names = get_pk_names(session, table_full_name)
13    method = get_process_method(session, table_full_name)
14
15    pk_err = check_pk_duplicates(session, batch_tbl, dsc_tbl,
    pk_names)
16    nn_err = check_not_null(session, table_full_name, batch_tbl,
    dsc_tbl)
17    dt_err = check_data_types(session, table_full_name, batch_tbl,
    cast_tbl, dsc_tbl)
18
19    load_log = load_data(session, table_full_name, cast_tbl,
    pk_names, method)
20
21    session.sql(f"DROP TABLE IF EXISTS {batch_tbl}").collect()
```

```
22 session.sql(f"DROP TABLE IF EXISTS {cast_tbl}").collect()
```

Listing 3.10: Orchestrazione DQ e caricamento finale

Per il caso di studio riportato in questo capitolo (receipt line), nel file procedure sono presenti anche le chiamate effettive dei tre step $RL \rightarrow QL \rightarrow BL$:

```
1 CALL DB_DEMO_FMK.RL_FMK.SP_RL_LOADING(
2   'DB_DEMO_RL.RL_TDM.TB_RECEIPT_LINE',
3   'DB_DEMO_RL.RL_TDM.STG_TB_RECEIPT_LINE',
4   'DB_DEMO_RL.RL_TDM.FF_CSV',
5   'CSV',
6   'manual'
7 );
8
9 CALL DB_DEMO_FMK.QL_FMK.SP_QL_LOADING(
10  'DB_DEMO_QL.QL_TDM.TB_RECEIPT_LINE',
11  'manual'
12 );
13
14 CALL DB_DEMO_FMK.BL_FMK.SP_BL_LOADING(
15  'DB_DEMO_BL.BL_TDM.TB_RECEIPT_LINE',
16  'manual'
17 );
```

Listing 3.11: Chiamate effettive del flusso TB_RECEIPT_LINE

Alla fine, dopo aver caricato tutto il dominio e le procedure, dalla UI di Snowflake è possibile osservare la struttura dei database e dei relativi schemi, riportata nella Figura 3.1:

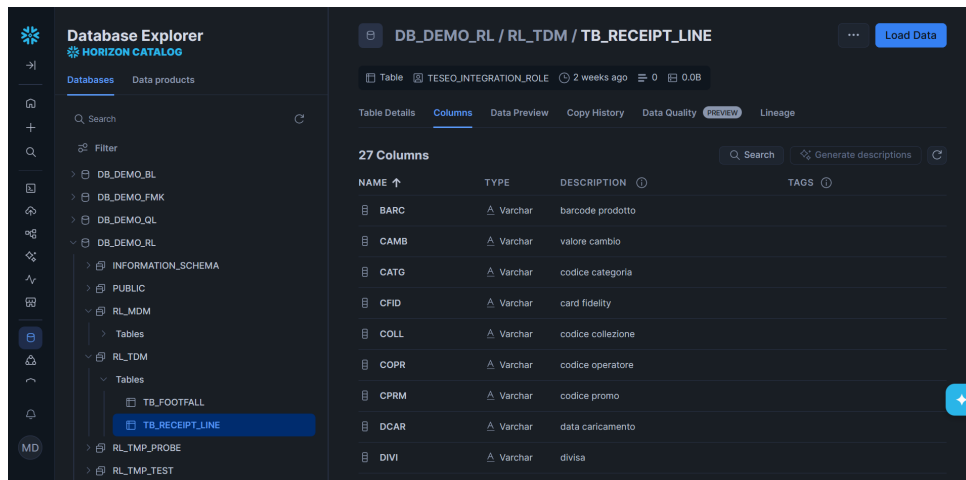


Figura 3.1: Struttura database e schemi in Snowflake

3.2 Configurazione DataHub

La piattaforma è stata avviata con DataHub Core v1.4.0 in ambiente Docker locale, con autenticazione del metadata service abilitata sia al servizio GMS sia al frontend:

```
1 datahub-gms-quickstart:
2   environment:
3     METADATA_SERVICE_AUTH_ENABLED: 'true'
4
5 frontend-quickstart:
6   environment:
7     METADATA_SERVICE_AUTH_ENABLED: 'true'
```

Listing 3.12: Estratto Docker Compose DataHub (auth abilitata)

Il processo completo di ingestion è stato eseguito in un ambiente virtuale python al fine di ottenere un comportamento isolato e ripetibile, con le dipendenze necessarie per i connettori Snowflake e dbt, ma senza errori di compatibilità o versioni. Di seguito i comandi principali di bootstrap dell'ambiente e avvio del quickstart (che usa il docker-compose locale personalizzato con authN abilitata):

```
1 python -m venv venv
2 venv\Scripts\Activate
3 python -m pip install --upgrade pip wheel setuptools
4 pip install acryl-datahub dbt-core dbt-snowflake
5 pip install "acryl-datahub[snowflake]"
6
7 datahub docker quickstart --quickstart-compose-file .\docker-
  compose.yml --version v1.4.0
8 datahub init
```

Listing 3.13: Comandi principali di bootstrap

3.3 Ingestion Snowflake

Ora che sia Snowflake sia DataHub sono configurati, sono passato alla configurazione dell'ingestion mediante `recipe.yml`. In particolare, ho deciso di limitare la configurazione al perimetro dei database demo e degli schemi funzionali usati, includendo sia le tabelle sia le viste e attivando la possibilità di estrarre il lineage a livello di colonna.

Per quanto riguarda le **credenziali** di accesso (e i secrets in generale), ho adottato una strategia più sicura di non hardcodare nulla nella recipe, ma di usare **variabili d'ambiente** che vengono caricate da un file `.env` dedicato. In

questo modo, la recipe è completamente agnostica rispetto all'ambiente e può essere facilmente riutilizzata o condivisa senza rischi di esposizione credenziali.

La **recipe completa** per estrarre i metadati solo da Snowflake è la seguente:

```
1 pipeline_name: snowflake_prod_main
2
3 source:
4   type: snowflake
5   config:
6     username: "${SNOWFLAKE_USERNAME}"
7     password: "${SNOWFLAKE_PASSWORD}"
8     account_id: "${SNOWFLAKE_ACCOUNT_ID}"
9     role: "${SNOWFLAKE_ROLE}"
10    warehouse: "${SNOWFLAKE_WAREHOUSE}"
11    platform_instance: "${SNOWFLAKE_PLATFORM_INSTANCE}"
12
13    database_pattern:
14      allow:
15        - DB_DEMO_RL
16        - DB_DEMO_QL
17        - DB_DEMO_BL
18        - DB_DEMO_FMK
19
20    schema_pattern:
21      allow:
22        - DBT
23        - RL_TDM
24        - RL_MDM
25        - QL_TDM
26        - QL_TDM_DSC
27        - QL_MDM
28        - QL_MDM_DSC
29        - BL_TDM
30        - BL_TDM_DSC
31        - BL_MDM
32        - BL_MDM_DSC
33        - META
34        - RL_FMK
35        - QL_FMK
36        - BL_FMK
37
38    include_tables: true
39    include_views: true
40    include_column_lineage: true
41    include_usage_stats: false
42
43    profiling:
44      enabled: false
```

```
45     stateful_ingestion:
46         enabled: false
47
48 sink:
49     type: datahub-rest
50     config:
51         server: "${DATAHUB_GMS_URL}"
52         token: "${DATAHUB_TOKEN}"
53         timeout_sec: 120
54         retry_max_times: 5
55         max_threads: 1
```

Listing 3.14: Recipe Snowflake

Riporto qui sotto il significato operativo di alcune delle opzioni che hanno avuto un ruolo fondamentale all'interno della mia recipe, in particolare per quanto riguarda la qualità del grafo di lineage e la stabilità dell'ingestion:

- `include_column_lineage:true`: abilita il lineage con granularità a livello di colonna quando il parser SQL riesce a ricostruire le dipendenze; è l'opzione chiave per passare da un lineage solo tabellare a un lineage utile per impact analysis;
- `include_usage_stats:false`: disabilita la raccolta delle statistiche di utilizzo (query history/usage) durante l'ingestion. Nel mio setup iniziale ho privilegiato stabilità e perimetro, rinviando l'enrichment con usage a una fase successiva;
- `profiling.enabled:false`: evita la profilazione dei dati (cardinalità, *null ratio*...), riducendo quindi tempo, costo computazionale e permessi richiesti (come detto sopra, ho dato precedenza alla stabilità e al perimetro);
- `stateful_ingestion.enabled:false`: evita il meccanismo di soft-delete degli oggetti non più trovati dalla sorgente. Ho mantenuto disattivata questa opzione nella recipe principale per non rischiare rimozioni indesiderate durante i primi run esplorativi;
- `database_pattern.allow` e `schema_pattern.allow`: restringono il perimetro ai soli database e schemi di progetto, riducendo il rumore nel catalogo e i tempi di scansione.

L'esecuzione può avvenire tramite comandi singoli, ma ho deciso di scrivere anche un **wrapper PowerShell** `ingest.ps1` che carica automaticamente le variabili da `.env` e lancia il comando di ingestion. Questa scelta è stata fatta per semplificare l'esecuzione ricorrente, ridurre il rischio di errori di digitazione ad ogni comando e rendere più **modulare** l'orchestrazione, in modo da poter estendere facilmente il wrapper per includere anche nei prossimi passaggi l'ingestion dbt e Power BI:

```

1 param(
2   [switch]\$IncludeDbt,
3   [switch]\$IncludePowerBIMock,
4   [switch]\$SkipSnowflake,
5   [switch]\$ReconcileDbtSnowflakeBridge,
6   [string]\$SnowflakeRecipe = 'venv\Ingestion\recipe_snowflake.yml',
7   [string]\$DbtRecipe = 'venv\Ingestion\recipe_dbt.yml',
8   [string]\$SnowflakeBridgeRecipe = 'venv\Ingestion\
9   recipe_snowflake_dbt_bridge.yml'
10 )
11 # Load .env into process scope
12 Get-Content \$envFile | Where-Object { \$_ -match '=' } | ForEach-
13   Object {
14     \$_name, \$_value = \$_ -split '=', 2
15     [Environment]::SetEnvironmentVariable(\$_name, \$_value, 'Process')
16   }
17 datahub ingest -c \$snowflakeRecipePath
18 if (\$IncludeDbt) { datahub ingest -c \$dbtRecipePath }
19 if (\$ReconcileDbtSnowflakeBridge) { datahub ingest -c \
    \$bridgeRecipePath }

```

Listing 3.15: Estratto orchestratore ingest.ps1

3.4 Ingestion dbt e raccordo con Snowflake

Dopo la pubblicazione del perimetro Snowflake, ho introdotto **dbt** per rendere esplicita la trasformazione logica (modello semantico) e ottenere un lineage più leggibile tra sorgenti, layer intermedi e mart finali. Operativamente, il lavoro è stato diviso in due filoni principali: costruzione artefatti dbt affidabili e configurazione della recipe DataHub (con filtri anti-rumore come visti precedentemente nella recipe Snowflake).

3.4.1 Setup progetto dbt e generazione artefatti

Il progetto dbt è stato organizzato fin da subito separando chiaramente:

- **full_coverage**: modelli auto-generati al fine di coprire in modo esteso gli oggetti Snowflake;
- **semantic layer**: modelli curati per il flusso analitico ($\text{stg} \rightarrow \text{int} \rightarrow \text{mart}$).

La scelta di materializzare **stg** e **int** come viste e **mart** come tabelle è stata fatta per trovare un trade-off tra tracciabilità e performance. Il profilo dbt punta a Snowflake e pubblica i risultati nello schema DBT.

Le sorgenti dichiarate invece coprono RL/QL/BL, aree DSC (discard) e meta-framework; in questo modo i modelli dbt possono referenziare sia dati operativi sia tabelle di controllo qualità, mantenendo continuità di lineage.

La pipeline dbt utilizzata per produrre artefatti coerenti con l'ingestion DataHub è la seguente:

```
1 dbt clean
2 dbt deps
3 dbt build --no-partial-parse
4 dbt docs generate --no-partial-parse
```

Listing 3.16: Build dbt e artefatti per ingestion

Gli artefatti `manifest.json`, `catalog.json` e `run_results.json` vengono poi referenziati dalla recipe dbt.

Nel flusso di comandi sopra mostrato, `dbt build --no-partial-parse` e `dbt docs generate --no-partial-parse` sono stati eseguiti in modo intenzionale per evitare effetti collaterali di cache durante l'evoluzione del progetto: in questo modo gli artefatti usati da DataHub sono sempre allineati allo stato reale del codice.

3.4.2 Recipe dbt e controllo del rumore nel grafo

La `recipe_dbt.yml` è configurata per:

- target platform **snowflake**;
- lineage a livello colonna (`include_column_lineage:true`);
- allineamento URN dbt/Snowflake (platform instance e conversione lowercase);
- inclusione selettiva dei soli modelli semantici **stg_**, **int_**, **mart_** per ridurre entità non utili (*rumore*).

Questa configurazione risolve quindi tre problemi tipici dell'ingestion dbt:

- **allineamento URN**: con `target_platform_instance` allineato a Snowflake, i nodi dbt vengono agganciati correttamente agli oggetti fisici già ingestionati, evitando duplicazioni e disconnessioni nel grafo;
- **riduzione del rumore**: il filtro `node_name_pattern` limita il grafo ai modelli semanticamente rilevanti, evitando saturazione con entità tecniche inutili al fine del lineage;

- **coerenza incrementale:** `stateful_ingestion` nella configurazione dbt consente di rimuovere metadati stantii quando cambiano nomi o modelli (evitando quindi discrepanze tra i metadati).

```

1 pipeline_name: dbt_prod_main
2
3 source:
4   type: dbt
5   config:
6     manifest_path: "${DBT_MANIFEST_PATH}"
7     catalog_path: "${DBT_CATALOG_PATH}"
8     run_results_paths:
9       - "${DBT_RUN_RESULTS_PATH}"
10    target_platform: "snowflake"
11    env: "${DBT_ENVIRONMENT}"
12    platform_instance: "${DBT_PLATFORM_INSTANCE}"
13    target_platform_instance: "${SNOWFLAKE_PLATFORM_INSTANCE}"
14    include_column_lineage: true
15    convert_column_urns_to_lowercase: true
16    infer_dbt_schemas: true
17    include_database_name: true
18    dbt_is_primary_sibling: false
19    node_name_pattern:
20      allow:
21        - "^model\\.teseo_dwh\\.(stg_|int_|mart_).*[\\$]"
22        - "^source\\.teseo_dwh\\.\\.\\.*[\\$]"
23    stateful_ingestion:
24      enabled: true
25      remove_stale_metadata: true
26      fail_safe_threshold: 100.0
27
28 sink:
29   type: datahub-rest
30   config:
31     server: "${DATAHUB_GMS_URL}"
32     token: "${DATAHUB_TOKEN}"
33     timeout_sec: 120
34     retry_max_times: 5
35     max_threads: 1

```

Listing 3.17: Recipe dbt

3.4.3 Riconciliazione bridge Snowflake-dbt

Per rimuovere eventuali incongruenze nel solo schema fisico `DB_DEMO_BL.DBT`, ho usato anche una terza recipe dedicata `recipe_snowflake_dbt_bridge.yml`, con `stateful_ingestion` abilitata e scope volutamente ristretto:

```

1 pipeline_name: snowflake_dbt_bridge_reconcile
2 source:
3   type: snowflake
4   config:
5     ...
6     include_tables: true
7     include_views: true
8     include_column_lineage: true
9     stateful_ingestion:
10      enabled: true
11      remove_stale_metadata: true
12 sink:
13   type: datahub-rest
14   config:
15     server: "${DATAHUB_GMS_URL}"

```

Listing 3.18: Recipe bridge Snowflake-dbt

L'orchestrazione completa Snowflake + dbt + bridge viene eseguita con un'unica invocazione (`.\ingest.ps1 -IncludeDbt -ReconcileDbtSnowflakeBridge`). Il risultato di questa esecuzione è osservabile nella Figura 3.2: l'immagine mostra il lineage a livello di colonna ottenuto dopo l'ingestion combinata di Snowflake e dbt, in cui ogni colonna dei modelli dbt risulta collegata alle colonne fisiche Snowflake da cui deriva, a conferma della corretta riconciliazione degli URN tra i due connettori.

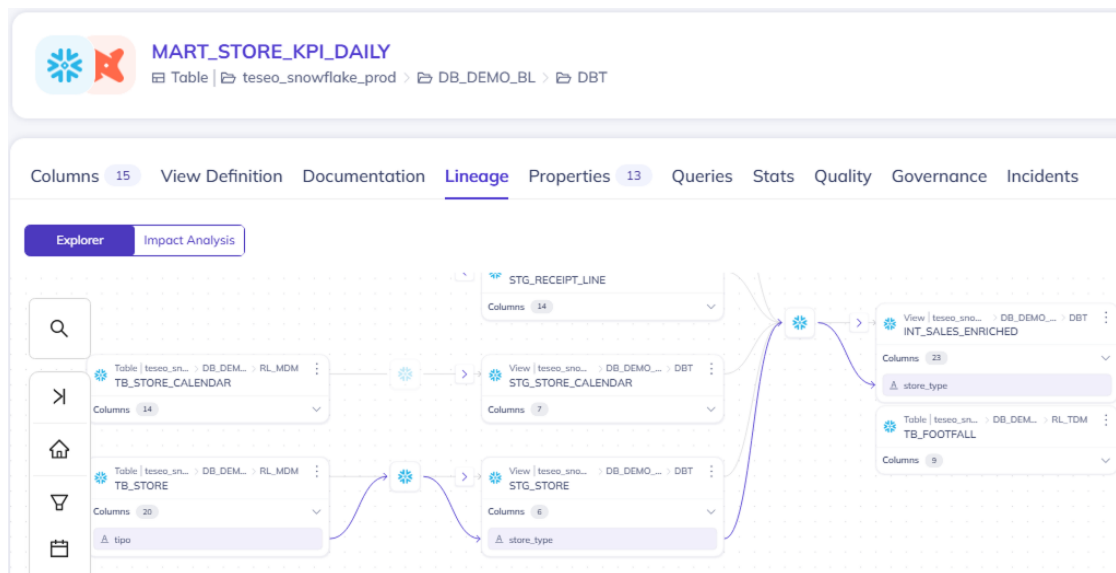


Figura 3.2: Risultato lineage dopo ingestion Snowflake + dbt

3.5 Ingestion Power BI: da mock locale a service

Nel progetto Teseo non era disponibile un semantic model Power BI già pronto da utilizzare come base di ingestion. Per questo ho costruito una struttura mock locale con lo stesso obiettivo informativo del caso reale: pubblicare sul GMS di Teseo il workspace, il semantic model, i report e il lineage sempre a livello di colonna/campo.

Come sempre, l'approccio è stato progettato per essere ripetibile e verificabile: prima generazione delle strutture mock, poi creazione del piano di ingestion, infine emissione verso DataHub con controlli automatici di validazione post-run.

3.5.1 Motivazione del modello locale

Una domanda lecita potrebbe essere: perché costruire un mock locale invece di costruire direttamente il semantic model e i report su un Service reale di Power BI? La motivazione principale è stata tecnica: senza un modello Power BI pre-esistente, il mock ha permesso di costruire velocemente e testare il tratto finale del lineage (dbt → semantic model BI) senza dipendere subito da tenant, workspace e permessi del Service, e quindi senza caricare strutture di prova in un ambiente di produzione.

Il mock locale si compone principalmente di due artefatti centrali:

- `powerbi_mock_entities_dbt.yaml` e la versione *full*: definiscono workspace, semantic models, tabelle, colonne, report, pagine e visual;
- `powerbi_lineage_map_dbt.csv` e la versione *full*: definiscono il legame esplicito tra colonne Power BI e upstream dbt (con supporto a `upstream_urn` per il riferimento univoco).

Questa scelta ha risolto due esigenze pratiche del mio caso: evitare mismatch sui nomi a causa della creazione manuale delle strutture e ottenere subito un fine-grained lineage consistente con i modelli dbt già pubblicati in precedenza (senza doverli modificare).

Un estratto del semantic model mock mostra che la struttura è già organizzata come un vero e proprio modello BI:

```

1 project:
2   name: Teseo Snowflake dbt Power BI Mock Full
3 powerbi:
4   platform: powerbi
5   platform_instance: local_desktop
6   workspace:
7     id: ws_teseo_uat

```

```

8     name: Teseo UAT
9   semantic_models:
10     - name: dbt_full_semantic
11       tables:
12         - name: AUTOGEN__DB_DEMO_BL__BL_MDM__TB_LOYALTY_CARD
13           columns: [TS_INGESTION, LOYALTY_CARD_ID, CUSTOMER_ID,
CARD_TYPE]

```

Listing 3.19: Estratto entities Power BI da dbt catalog

Il mapping CSV completa il modello con il legame colonna-colonna verso upstream dbt:

```

1 workspace_id,report_name,pbi_dataset_name,pbi_table_name,
  pbi_column_name,upstream_platform,upstream_dataset,
  upstream_column,upstream_urn
2 ws_teseo_uat,DBT Business Semantic Dashboard,dbt_full_semantic,
  AUTOGEN__DB_DEMO_BL__BL_MDM__TB_LOYALTY_CARD,LOYALTY_CARD_ID,
  dbt,teseo_dbt_semantic.DB_DEMO_BL.DBT.
  autogen_db_demo_bl__bl_mdm__tb_loyalty_card,LOYALTY_CARD_ID,"
  urn:li:dataset:(urn:li:dataPlatform:dbt,teseo_dbt_semantic.
  DB_DEMO_BL.DBT.autogen_db_demo_bl__bl_mdm__tb_loyalty_card,
  PROD)"

```

Listing 3.20: Estratto mapping Power BI mock a dbt

3.5.2 Script di generazione e ingestion

La pipeline mock è stata implementata con script Python e PowerShell che separano chiaramente generazione, ingestion e verifica. In particolare:

- `generate_powerbi_from_dbt.py`: legge il `catalog.json` dbt e genera automaticamente le entità YAML dal catalogo e il mapping CSV relativo, con logica di naming coerente con il modello reale (usando il prefisso `AUTOGEN__` per evidenziare le tabelle generate automaticamente);
- `ingest_mock_powerbi.py`: costruisce `powerbi_mock_plan.json` e, in modalità reale, emette eventi MCP verso DataHub contenenti schema, lineage dataset-level e lineage fine-grained;
- `ingest_mock_powerbi_dbt_full.ps1`: file di orchestrazione che esegue la generazione automatica da dbt e poi l'ingestion, con opzione `-DryRun` per validare la generazione senza emissione (utile a fini di debug);
- `ingest_mock_powerbi_bundle.ps1`: combina due run (full semantic + mart-focused) per coprire scenari di reporting diversi.

```

1 # verifica piano senza invio
2 .\powerbi_mock\scripts\ingest_mock_powerbi_dbt_full.ps1 -DryRun
3
4 # ingestion reale del semantic model full
5 .\powerbi_mock\scripts\ingest_mock_powerbi_dbt_full.ps1
6
7 # bundle completo: 2 report full + 1 report mart
8 .\powerbi_mock\scripts\ingest_mock_powerbi_bundle.ps1

```

Listing 3.21: Esecuzione pipeline Power BI mock

Come anticipato, lo script di ingestion in particolare include due funzioni molto utili per ridurre gli errori reali di integrazione:

- auto-risoluzione degli `upstream_urn` contro DataHub;
- auto-risoluzione degli `upstream_column` rispetto ai `fieldPath` effettivi nello schema metadata.

Queste opzioni rendono il mock molto robusto quando cambiano naming tra dbt, Snowflake e PBI.

3.5.3 Connessione a Power BI Service

Dopo la validazione del flusso *mock*, nel progetto ho attivato anche l'ingestion su Power BI Service reale, mantenendo la stessa impostazione di robustezza operativa: credenziali esterne alla recipe, test preliminare di connettività e perimetro ristretto al workspace selezionato.

La recipe che permette di *ingestionare* il layer Power BI abilita l'estrazione di report, dashboard, schema dataset e lineage (anche *fine-grained* fin dove possibile), con mapping del datasource Snowflake verso la stessa *platform instance* già usata nel perimetro Snowflake e dbt:

```

1 pipeline_name: powerbi_real_main
2
3 source:
4   type: powerbi
5   config:
6     tenant_id: "${POWERBI_TENANT_ID}"
7     client_id: "${POWERBI_CLIENT_ID}"
8     client_secret: "${POWERBI_CLIENT_SECRET}"
9
10    workspace_id_pattern:
11      allow:
12        - "^${POWERBI_WORKSPACE_ID}$"
13
14    workspace_type_filter:

```

```

15     - "Workspace"
16
17     env: "${POWERBI_ENV}"
18     platform_instance: "${POWERBI_PLATFORM_INSTANCE}"
19     extract_reports: true
20     extract_dashboards: true
21     extract_dataset_schema: true
22     extract_lineage: true
23     extract_column_level_lineage: true
24     convert_lineage_urns_to_lowercase: true
25
26     server_to_platform_instance:
27       "${POWERBI_SNOWFLAKE_SERVER}":
28         platform_instance: "${SNOWFLAKE_PLATFORM_INSTANCE}"
29         env: "${POWERBI_ENV}"
30
31   sink:
32     type: datahub-rest
33     config:
34       server: "${DATAHUB_GMS_URL}"
35       token: "${DATAHUB_TOKEN}"

```

Listing 3.22: Recipe Power BI Service

Questa fase nel progetto è risultata funzionante e ha confermato la possibilità di pubblicare metadati e lineage anche dal Service reale [5, 6].

Successivamente, nel percorso evolutivo di Teseo, questa modalità è stata progressivamente affiancata da un approccio generativo: data la buona riuscita della ingestion Power BI, si è deciso di passare a sviluppare una soluzione in cui il frontend Power BI viene creato mediante gli strumenti MCP (riducendo i passaggi manuali e rendendo più coerente l'allineamento tra metadato tecnico e livello di consumo analitico) e pubblicato come un Power BI custom costruito *ad hoc* a partire dal lineage completo e in particolare dal livello *ingestionato* più vicino all'utente finale [18, 19].

3.6 Costruzione del lineage end-to-end

Con la catena Snowflake → dbt → Power BI, il grafo risultante mostra un lineage end-to-end navigabile dallo strato fisico al livello di consumo. Questo passaggio è quello che rende Teseo particolarmente potente: non solo catalogazione, ma **tracciabilità completa** pronta per impact analysis, troubleshooting e ispezione automatizzata.

Dal punto di vista pratico, l'osservabilità end-to-end abilita tre capacità chiave:

- **analisi di impatto immediata:** da una colonna sorgente è possibile risalire a tutti gli oggetti downstream (fino ai modelli semantici, report e visual);
- **governance semantica coerente:** ownership, tag, glossary terms e descrizioni restano allineati lungo l'intera catena di trasformazione e possono anche essere propagati o arricchiti in ogni punto del flusso;
- **accelerazione delle automazioni:** grazie al lineage completo, gli strumenti MCP di Teseo possono proporre o applicare update mirati con contesto già validato e con scope ampio su tutto il grafo dei metadati.

Un percorso di *fine-grained lineage* tipico osservabile nel grafo è:

1. tabella sorgente in Snowflake;
2. modello dbt con lineage di trasformazione;
3. semantic model, report e visual Power BI con dipendenze colonnari esplicite.

La Figura 3.3 mostra un esempio di questa catena completa così come appare nel grafo di Teseo:

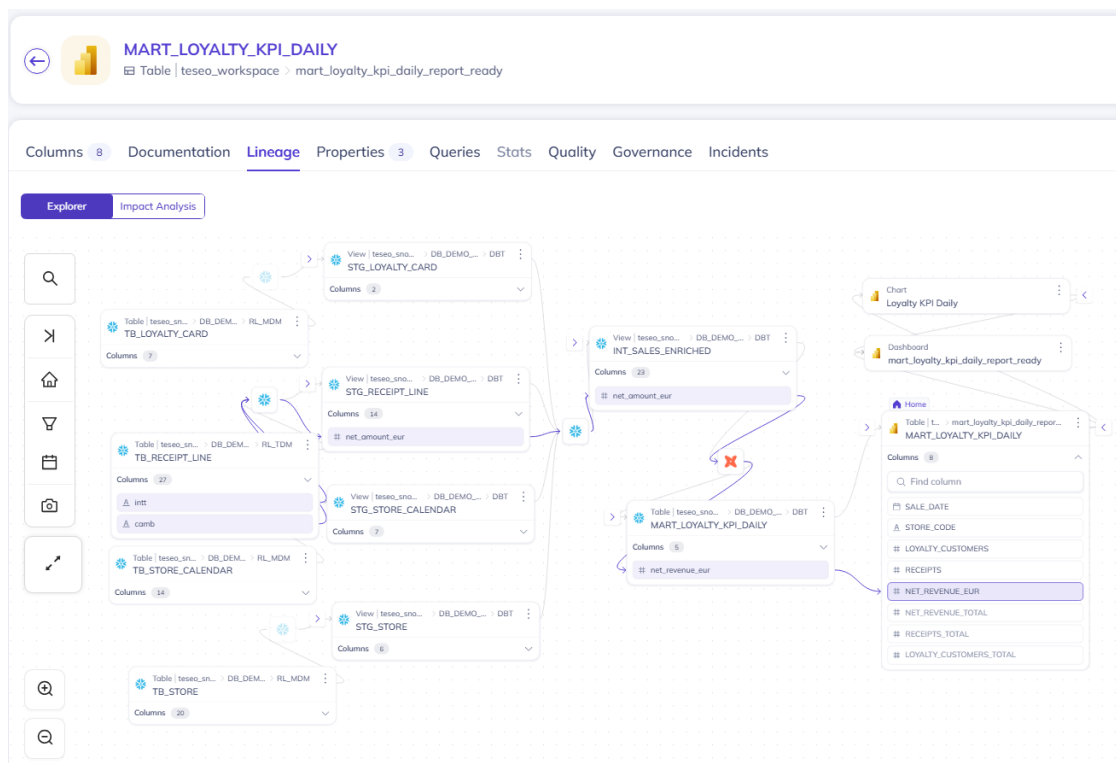


Figura 3.3: Lineage end-to-end in Teseo

3.7 Esempio di ingestion da altre sorgenti

Il pattern implementato in Teseo (ricette + validazioni + mapping coerente degli URN + controllo del rumore) è riusabile anche per altre sorgenti, mantenendo lo stesso obiettivo: costruire lineage utile a governance e automazione, non solo un enorme glossario tecnico [5, 9].

3.7.1 Caso Databricks (Unity Catalog)

Databricks è un altro ottimo caso d'esempio perché combina metastore, notebooks, job activity e lineage. Nella documentazione ufficiale di DataHub, la sorgente `databricks` (*Unity Catalog*) espone già di suo capacità native su schema, ownership, usage e lineage (incluse opzioni per *column-level lineage*) [20].

Una recipe minimale a scopo dimostrativo è la seguente:

```

1 pipeline_name: databricks_uc_main
2
3 source:
4   type: databricks
5   config:
6     workspace_url: "${DATABRICKS_WORKSPACE_URL}"
7     client_id: "${DATABRICKS_CLIENT_ID}"
8     client_secret: "${DATABRICKS_CLIENT_SECRET}"
9     platform_instance: "${DATABRICKS_PLATFORM_INSTANCE}"
10    env: "${DATABRICKS_ENV}"
11
12    include_tables: true
13    include_views: true
14    include_table_lineage: true
15    include_column_lineage: true
16    include_usage_statistics: true
17    include_ownership: true
18
19    warehouse_id: "${DATABRICKS_WAREHOUSE_ID}"
20
21    catalog_pattern:
22      allow:
23        - "^main$"
24        - "^analytics$"
25
26    stateful_ingestion:
27      enabled: true
28      remove_stale_metadata: true
29      fail_safe_threshold: 100.0
30
31 sink:
```

```
32 type: datahub-rest
33 config:
34     server: "${DATAHUB_GMS_URL}"
35     token: "${DATAHUB_TOKEN}"
```

Listing 3.23: Recipe Databricks

In questo scenario, il lineage osservabile è leggibile come una catena **bronze** -> **silver** -> **gold** -> **BI**: una tabella raw su catalog **main**, una vista o tabella curata su **analytics.silver**, una tabella **analytics.gold** e infine un livello di consumo (dashboard o semantic model). L'effetto pratico è analogo a quanto ottenuto in Teseo: da una singola colonna business è possibile risalire alla sorgente e viceversa, con forte supporto a impact analysis.

3.7.2 Il vero valore di Teseo

Questo esempio mostra come Teseo sia progettualmente estendibile: una volta fissata la ricetta adatta al caso d'uso e il controllo qualità del lineage, l'infrastruttura può incorporare nuove sorgenti con sforzo incrementale e con ritorno immediato in termini di osservabilità e governance.

In altre parole, la potenza di un tool come Teseo non sta solo nel singolo connettore usato oggi, ma nella capacità di **riusare** lo stesso framework su domini e piattaforme **eterogenei** mantenendo una vista unificata end-to-end del metadato.

Capitolo 4

Data Governance

Questo capitolo descrive come Teseo abilita in modo operativo la **Governance** del patrimonio informativo aziendale: infatti, dopo aver consolidato ingestion e lineage multi-source, il passo successivo è rendere i metadati ottenuti governabili, standardizzati e verificabili nel tempo.

Nel contesto del progetto, la governance non è stata trattata come un layer accessorio, ma come una **componente strutturale** della piattaforma: classificazione degli asset, responsabilità, tassonomia business, controllo qualità e auditabilità sono tutti aspetti che sfruttano lo stesso grafo metadati già costruito nei capitoli precedenti [9, 14, 17].

4.1 Data Governance e metadati

4.1.1 Cos'è la Data Governance

Con **Data Governance** si intende l'insieme di principi, ruoli, processi e controlli con cui un'organizzazione assicura che i dati siano [21, 22]:

- **affidabili** (accurati, completi e aggiornati);
- **comprensibili** (documentati, contestualizzati e semanticamente coerenti);
- **responsabilizzati** (owner espliciti, processi di validazione e accountability);
- **conformi** (allineati a vincoli normativi e policy interne);
- **riusabili** (discovery efficace e basso attrito all'interno del team).

In altre parole, la governance non è una funzionalità isolata, ma un sistema continuo di decisione e controllo sul ciclo di vita del dato.

Nel contesto moderno, la governance opera soprattutto sulla base dei **metadati**: senza metadati non è possibile sapere chi possiede un asset, quale significato business abbia, come la qualità evolve nel tempo e quali dipendenze produca a valle.

Per questo motivo, Teseo adotta una strategia *metadata-first*: ingestion, lineage e governance condividono lo stesso modello e lo stesso identificatore (URN), permettendo un controllo unificato sugli asset [14].

4.1.2 Dimensioni principali della governance

Nel perimetro di Teseo, la governance viene implementata su più dimensioni complementari:

- **semantica** (*knowledge*): glossario, descrizioni e structured properties;
- **organizzazione** (*ownership*): ownership, domini, gruppi e ruoli;
- **qualità** (*quality*): assertions, monitoraggi e incidenti;
- **sicurezza** (*security*): workflow, sicurezza e automazioni (anche da API/CLI);
- **compliance e accesso** (*accessibility*): classificazioni, policy e approvazioni.

La Figura 4.1 riassume queste cinque dimensioni, riportando per ciascuna i meccanismi concreti con cui Teseo le implementa.

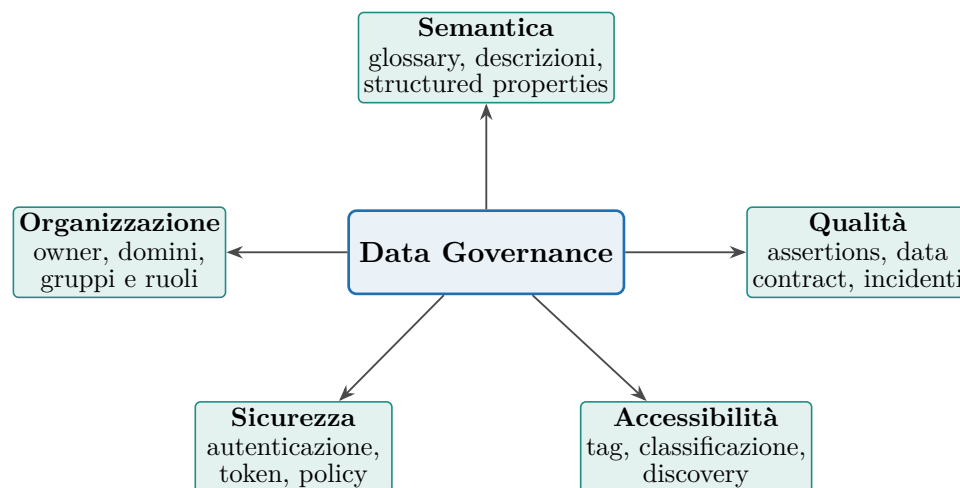


Figura 4.1: Data Governance in Teseo

4.2 Ruolo della governance in Teseo

In Teseo, DataHub Core svolge il ruolo di *control plane* dei metadati: ogni asset (dataset, colonna, dashboard...) viene descritto da aspetti semanticamente tipizzati e quindi può essere soggetto a comuni regole di governance.

I benefici principali di questo approccio sono:

- **standardizzazione:** tag, domini, ownership e glossario seguono convenzioni condivise;
- **tracciabilità:** ogni modifica ai metadati è riconducibile a entità e URN specifiche;
- **unificazione:** la governance insiste sugli stessi oggetti già usati per ingestion e lineage;
- **scalabilità:** i processi possono essere gestiti da UI, CLI, GraphQL o SDK con un modello coerente.

4.3 Entità di Governance in Teseo

4.3.1 Tagging e classificazione

I **tag** permettono di classificare rapidamente e flessibilmente asset e colonne. Per esempio, sono utilizzabili per distinguere entità sensibili (es. PII), dataset certificati per reporting, layer tecnici o aree a rischio qualitativo.

Si può aggiungere un tag ad un dataset tramite UI o anche tramite API GraphQL, ad esempio per indicare che è deprecato:

```
1 mutation addTags {  
2   addTags(  
3     input: {  
4       tagUrns: ["urn:li:tag:deprecated"],  
5       resourceUrn: "urn:li:dataset:(urn:li:dataPlatform:hive,  
6         fct_users_created,PROD)"  
7     }  
8   )  
}
```

Listing 4.1: Aggiunta di tag ad un dataset mediante GraphQL

Analogamente, il tag può essere rimosso quando non è più valido:

```
1 mutation removeTag {
2   removeTag(
3     input: {
4       tagUrn: "urn:li:tag:deprecated",
5       resourceUrn: "urn:li:dataset:(urn:li:dataPlatform:hive,
6         fct_users_created,PROD)"
7     }
8   )
9 }
```

Listing 4.2: Rimozione di tag da un dataset mediante GraphQL

4.3.2 Domini e ownership

La governance su Teseo comprende anche l'uso di **domini** i quali permettono di organizzare e categorizzare gli asset in base al contesto di appartenenza (es. Sales, Finance, Marketing...), migliorandone quindi discovery e accountability. Parallelamente, l'**ownership** rende esplicita la responsabilità tecnica e business sui vari asset, in modo da facilitarne la gestione e la tracciabilità.

Come per i tag, anche domini e ownership possono essere gestiti sia da UI sia da API. Qui sotto un esempio di assegnazione di un dominio ad un dataset tramite GraphQL:

```
1 mutation setDomain {
2   setDomain(
3     domainUrn: "urn:li:domain:marketing",
4     entityUrn: "urn:li:dataset:(urn:li:dataPlatform:hive,
5       fct_users_created,PROD)"
6   )
7 }
```

Listing 4.3: Assegnazione dominio ad un dataset mediante GraphQL

Anche l'ownership può essere assegnata tramite GraphQL, ad esempio per indicare l'owner tecnico di un dataset:

```
1 mutation addOwner {
2   addOwner(
3     input: {
4       ownerUrn: "urn:li:corpuser:data.engineer",
5       resourceUrn: "urn:li:dataset:(urn:li:dataPlatform:hive,
6         fct_users_created,PROD)",
7       ownerEntityType: CORP_USER,
8       type: TECHNICAL_OWNER
9     }
10  }
```

```

9   )
10  }

```

Listing 4.4: Assegnazione owner tecnico mediante GraphQL

4.3.3 Business Glossary e Structured Properties

Per allineare linguaggio tecnico e linguaggio business, esistono due componenti con ruoli complementari:

- **Glossary Terms:** definizioni condivise e riusabili di concetti *business*;
- **Structured Properties:** attributi tipizzati e validabili per estendere la semantica degli asset.

Nel modello operativo, i glossari permettono la riduzione delle ambiguità interpretative, mentre le structured properties permettono la creazione di policy più precise (ad esempio criticità, classificazione, livello di servizio...).

Qui sotto un esempio di aggiunta di un glossary term a un dataset tramite GraphQL:

```

1  mutation addTerms {
2    addTerms(
3      input: {
4        termUrns: ["urn:li:glossaryTerm:finance.revenue"],
5        resourceUrn: "urn:li:dataset:(urn:li:dataPlatform:hive,
6          fct_users_created,PROD)"
7      }
8    )
9  }

```

Listing 4.5: Aggiunta di un glossary term a dataset mediante GraphQL

4.3.4 Assertions e Data Contracts

La governance in Teseo non riguarda soltanto il puro aspetto di catalogazione, ma anche il poter dimostrare la qualità del catalogo metadati. Per fare ciò, Teseo supporta diverse tipologie di assertions (ovvero affermazioni verificabili su asset specifici):

- **column:** verifica che una colonna/campo rispetti certe condizioni (es. non null, cardinalità, distribuzione valori);
- **freshness:** verifica che un dataset sia aggiornato entro una finestra temporale attesa;

- **volume**: verifica che un dataset abbia un volume di record entro certi limiti (quindi in termini di *dimensioni*);
- **schema**: verifica che lo schema di un dataset rispetti una definizione attesa;
- **custom**: assertions personalizzate basate su query SQL scritte appositamente per verificare criteri specifici.

Oltre a ciò, Teseo permette la composizione di **Data Contracts** basati su assertion selezionate, in quanto un data contract è una combinazione di assertions che definiscono le aspettative su un asset (come un *pacchetto* di assertions).

Queste strutture di verifica consentono di passare da una documentazione puramente statica ad un modello dinamico **policy + verifica**, in cui le aspettative su un dataset diventano controlli eseguibili e monitorabili nel tempo mediante strumenti di monitoraggio e segnalazione.

Qui sotto un esempio di freshness assertion scritta tramite GraphQL:

```

1 mutation upsertDatasetFreshnessAssertionMonitor {
2   upsertDatasetFreshnessAssertionMonitor(
3     input: {
4       entityUrn: "urn:li:dataset:(urn:li:dataPlatform:snowflake,
5         analytics.orders,PROD)",
6       evaluationSchedule: {
7         timezone: "Europe/Rome",
8         cron: "0 */8 * * *"
9       },
10      mode: ACTIVE
11    }
12  ) {
13    urn
14  }

```

Listing 4.6: Freshness assertion mediante GraphQL

4.3.5 Incident management

Quando un assertion/contract fallisce o emerge un'anomalia non prevista, la piattaforma permette di aprire e tracciare **incidenti** associati a specifici asset: un incidente è un evento che indica un problema con uno specifico asset, segnalando quest'ultimo come non conforme. In termini di governance, questo riduce il tempo medio di diagnostica e migliora la comunicazione tra data owner, data engineer e stakeholder lato business.

Infatti, gli incidenti sono fortemente integrati con il framework di *assertions*: quando un controllo qualità configurato per generare incidenti fallisce, l'incidente viene creato automaticamente e mantiene il collegamento con la sorgente che lo ha generato (tramite `sourceUrn`) in modo da garantire la tracciabilità tra evento, regola violata e asset coinvolti, e da distinguere chiaramente incidenti *automatici* (da monitoraggio) e *manuali* (da segnalazione operativa) [23, 24].

Il modello di un incidente include tipicamente alcune proprietà come [24]:

- tipo (freshness, volume, schema...);
- priorità (CRITICAL, HIGH, MEDIUM, LOW);
- stato (ACTIVE, RESOLVED);
- eventuali assegnatari e lista di entità impattate (dataset, dashboard, chart...).

Sugli asset governabili (ovvero dataset, dashboard, chart, dataFlow, dataJob e schemaField), Teseo espone anche un **incidentsSummary** con conteggio degli incidenti attivi/risolti e breakdown per priorità (utile per avere un quadro generale in UI senza interrogare ogni singola entità) [24]. Come per ogni altra entità, la gestione degli incidenti è disponibile anche via GraphQL (`raiseIncident`, `updateIncident`, `updateIncidentStatus`), ma per creare o aggiornare incidenti su un asset è necessario avere il privilegio `EDIT_ENTITY_INCIDENTS` [17, 24]. Il ruolo principale degli incidenti è quello di fornire un indicatore di **health status** complessivo degli asset: la presenza di incidenti attivi ad alta priorità può marcare rapidamente un asset come non conforme, facilitando diagnostica, prioritizzazione e gestione.

La Figura 4.2 illustra il ciclo di vita di un incident nel progetto: un'assertion di freschezza sul layer Business che fallisce genera automaticamente un incident, mappato sull'asset di origine tramite `sourceUrn`, che viene poi preso in carico dall'owner fino alla risoluzione.

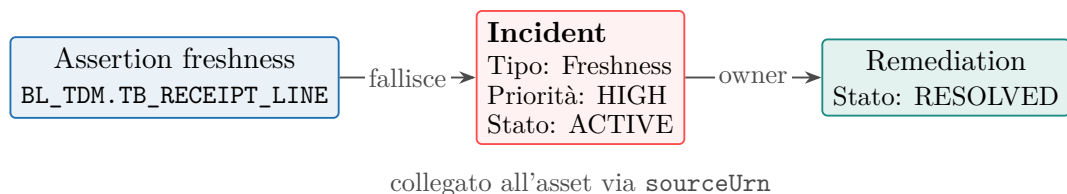


Figura 4.2: Ciclo di vita di un incident generato da un'assertion fallita

Capitolo 5

MCP Server e strumenti AI

Questo capitolo descrive l'implementazione e l'utilizzo di **server MCP (Model Context Protocol)** per abilitare operazioni assistite dagli agenti AI su Teseo, con focus su casi d'uso pratici e benefici in termini di produttività.

L'obiettivo non è soltanto introdurre un nuovo *canale* di integrazione, ma mostrare come un server MCP possa funzionare da *layer operativo* tra modelli linguistici, metadati di governance e automazioni [11]. In questo contesto, Teseo adotta due principali livelli complementari:

- strumenti MCP **read-oriented** basati sugli strumenti del server MCP ufficiale di DataHub Core (utili a fini di discovery, contesto e lineage);
- strumenti MCP **write-oriented** custom (utili a fini di remediation, arricchimento dei metadati e aggiornamenti controllati).

5.1 Model Context Protocol

MCP è un protocollo standard open-source creato per collegare applicazioni AI a sistemi esterni (dati, strumenti, workflow...) garantendo interoperabilità. L'idea centrale è ridurre la proliferazione di integrazioni ad hoc *client-sorgente* sostituendole con una semantica standard che consente la comunicazione tra AI e endpoints [7, 25, 26].

In termini architetturali, MCP adotta una separazione chiara tra:

- **host** (app AI che orchestra la conversazione e usa gli strumenti);
- **client MCP** (componente che mantiene la connessione verso il server);

- **server MCP** (processo che espone capability verso il client).

Questa separazione rende possibile usare lo stesso server da host differenti (IDE, chatbot, agenti...) potendo preservare lo stesso contratto operativo nonostante i diversi modi di interagire con esso [27, 28].

La Figura 5.1 mostra come la separazione host-client-server si concretizza in Teseo: l'host (un'applicazione come Claude Desktop o un IDE) racchiude il modello LLM e il client MCP e, tramite quest'ultimo, dialoga con due server, ovvero il server MCP ufficiale di DataHub (read-oriented) e il server custom sviluppato nel progetto (con i 17 strumenti di governance e gli 8 strumenti Power BI), entrambi collegati al GMS e quindi al catalogo dei metadati.

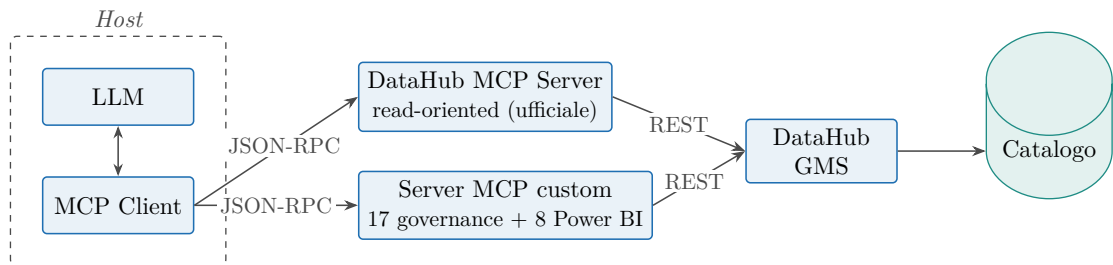


Figura 5.1: Architettura MCP in Teseo

5.1.1 Data layer e transport layer

Il protocollo MCP è organizzato in due layer principali [25, 27]:

- **Data layer:** protocollo stateful basato su JSON-RPC 2.0 con lifecycle management, capability negotiation e primitive;
- **Transport layer:** canali di comunicazione (tipicamente `stdio` per processi locali e `streamable HTTP` per server remoti).

Le primitive server-side più rilevanti sono:

- **tools** (*strumenti*): funzioni eseguibili dal modello (come query, update metadata, orchestrazione...);
- **resources** (*risorse*): contenuti contestuali (come documenti, schemi...);
- **prompts** (*prompt*): template riusabili per pattern conversazionali e task guidati.

Lato client, il protocollo include anche primitive utili per scenari avanzati (come sampling, logging...), fondamentali per agenti che richiedono cicli iterativi o feedback utente esplicito [25, 27].

La Figura 5.2 illustra i due layer del protocollo nel caso di Teseo: il *data layer* (logico, basato su JSON-RPC 2.0) viaggia *sopra* il *transport layer* (fisico) ed è attraverso il data layer che il server espone le proprie primitive (tools, resources e prompts).

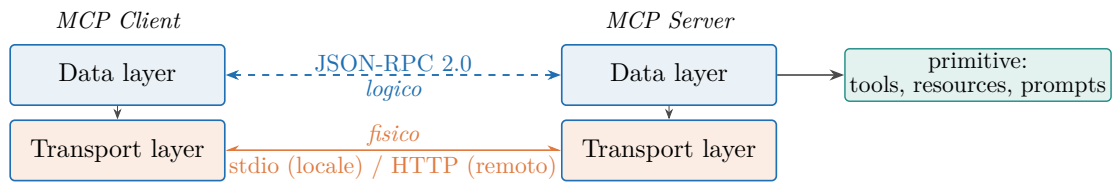


Figura 5.2: Data layer e transport layer

5.1.2 Lifecycle di sessione

Dato che l'obiettivo è garantire un'interazione standard e coerente, una sessione MCP tipica segue una sequenza prevedibile:

1. **initialize**: handshake iniziale e negoziazione della versione del protocollo;
2. **capability exchange**: dichiarazione delle feature disponibili;
3. **discovery**: elenco dinamico di strumenti, risorse e prompts disponibili;
4. **execution**: invocazioni degli strumenti con payload validato;
5. **notifications**: aggiornamento reattivo in caso di cambi di stato e capability.

Un esempio semplificato di richiesta di handshake MCP è il seguente:

```

1 {
2   "jsonrpc": "2.0",
3   "id": 1,
4   "method": "initialize",
5   "params": {
6     "protocolVersion": "2025-06-18",
7     "capabilities": {
8       "elicitation": {}
9     },
10    "clientInfo": {
11      "name": "teseo-agent-client",

```

```
12 |         "version": "1.0.0"
13 |     }
14 | }
15 | }
```

Listing 5.1: Handshake MCP semplificato

Come si può vedere anche dall'esempio di richiesta di handshake, l'adozione di un lifecycle esplicito e capability-based migliora la robustezza e l'osservabilità della soluzione: ogni invocazione di uno strumento avviene su un contratto noto, riducendo chiamate non previste e comportamenti non riproducibili [25, 27].

5.1.3 Sicurezza e trust boundaries

Il protocollo MCP abilita però anche azioni potenzialmente critiche (lettura di dati sensibili, esecuzione degli strumenti, scritture non controllate...), quindi richiede dei *guardrail* espliciti al fine di garantire un'adozione sicura e controllata, specialmente in ambienti enterprise:

- consenso e controllo da parte dell'utente sulle operazioni eseguite;
- principio di *least privilege* sugli strumenti e risorse esposti;
- forte logging e audit delle invocazioni e dei risultati, in modo da poter ricostruire ogni azione intrapresa dall'agente e intervenire in caso di comportamenti anomali;
- separazione degli ambienti dev/test/prod per limitare l'impatto di errori e l'esposizione di dati sensibili.

La stessa specifica ufficiale [25] sottolinea che il protocollo non può automaticamente imporre tutti i vincoli di sicurezza: l'implementazione concreta deve integrare policy e controlli applicativi, ovvero lo sviluppatore **DEVE**:

- costruire un robusto flusso di autorizzazioni nelle applicazioni che espongono strumenti MCP;
- fornire una documentazione chiara delle implicazioni di sicurezza;
- implementare controlli di accesso e protezioni dei dati;
- seguire le best practice di sicurezza nelle integrazioni;
- considerare la privacy nelle progettazioni delle funzionalità (*privacy by design*).

5.1.4 MCP in contesti data-centric

In contesti come Teseo, il valore del server MCP emerge in modo evidente in quanto fornisce strumenti per un'interazione potente e semplificata con il catalogo dei metadati. Infatti il modello può:

- cercare gli asset affidabili in ecosistemi eterogenei;
- capire la provenienza e gli impatti tramite analisi del lineage;
- recuperare la semantica business degli asset (owner, tag, glossary, domini);
- trasformare contesto in azioni operative (documentazione, classificazione, remediation...).

In assenza di un protocollo standard, ogni integrazione tende a essere fragile, costosa da mantenere e difficile da riusare. MCP riduce questa frammentazione fornendo un punto di interoperabilità unico [7, 26].

5.2 DataHub MCP Server

Il server MCP di DataHub espone ai client AI le operazioni principali eseguibili sul grafo dei metadati: ricerca strutturata, lineage, metadata retrieval e query intelligence. Come detto in precedenza, questo strumento rappresenta il layer standard per trasformare il catalogo in *contesto operativo* per agenti [18, 19].

5.2.1 Deployment

Secondo la documentazione ufficiale esistono due modalità principali d'uso [18]:

- **Managed MCP Server** (DataHub Cloud, endpoint gestito direttamente da DataHub);
- **Self-hosted MCP Server** (DataHub Core, esecuzione locale/remota controllata e configurabile dallo sviluppatore).

Nel caso di Teseo, che utilizza come base funzionale DataHub Core, il pattern adottato è pertanto quello self-hosted, con variabili di connessione verso GMS:

```
1 DATAHUB_GMS_URL=http://localhost:8080
2 DATAHUB_GMS_TOKEN=<personal-access-token>
3 TOOLS_IS_MUTATION_ENABLED=true
4 TOOLS_IS_USER_ENABLED=true
5 SEMANTIC_SEARCH_ENABLED=true
```

Listing 5.2: Variabili principali per DataHub MCP self-hosted

5.2.2 Catalogo degli strumenti

Gli strumenti del MCP server ufficiale compongono un set orientato maggiormente a discovery e analisi [18, 19]:

- **search** = ricerca strutturata con filtro SQL-like su dataset, campi, tag, glossary terms;
- **get_lineage** = recupero lineage a livello dataset e colonna;
- **get_dataset_queries** = recupero query rappresentative per dataset;
- **get_entities** = recupero dettagliato di entità (dataset, campo, tag, glossary term);
- **list_schema_fields** = recupero schema della tabella con tipi e descrizioni;
- **get_lineage_paths_between** = recupero percorso di lineage tra due entità.

Tramite gli ultimi aggiornamenti di DataHub Core, sono state aggiunte anche altre categorie di strumenti, ampliando le possibilità operative:

- **mutation tools**: operazioni di modifica su tag, terms, owner, domain, descrizioni e structured properties;
- **user tools**: strumenti per gestire l'identità autenticata (es. `get_me`);
- **document tools**: ricerca, filtraggio e salvataggio di documenti all'interno della knowledge base.

5.3 Strumenti write-oriented custom

Oltre agli strumenti ufficiali read-oriented, nel progetto sono stati implementati strumenti custom orientati alle operazioni di **scrittura** dei metadati su DataHub al fine di automatizzare mutation standard in flussi ripetitivi e di creare dei layer di semplificazione intermedi o verso l'utente finale.

La struttura è basata su **FastMCP** e **SDK DataHub** (`DatahubRestEmitter`, `MetadataChangeProposalWrapper`, `DataHubGraph`) con solita configurazione tramite variabili d'ambiente come `DATAHUB_GMS_URL` e `DATAHUB_GMS_TOKEN`.

5.3.1 Struttura del file

L'implementazione di questi strumenti write-oriented è composta di alcuni blocchi funzionali principali:

1. helper infrastrutturali (per connection, audit stamp e fetch schema esistente);
2. logica di merge (non distruttiva) sui metadati delle colonne;
3. strumenti per container e path;
4. strumenti per soft-delete e restore;
5. strumenti per gestire descrizioni, tag e glossary terms (*governance*);
6. strumenti per lineage table-level e column-level.

Questo disegno di architettura riesce bene nel suo intento di separare la parte **plumbing** dalla parte **domain operation** con il solito approccio di favorire modularità, manutenibilità e riuso (fondamentali in sistemi distribuiti come questo).

5.3.2 Merge non distruttivo

Come accennato, un punto chiave di questa strategia implementativa è la funzione di merge tra stato esistente e aggiornamenti richiesti (ovvero *stato corrente* e *stato desiderato*). L'approccio adottato **preserva** i campi non toccati e concatena tag e terms evitando duplicati (questo evita effetti collaterali frequenti nelle update massive, come per esempio la perdita involontaria di tags o glossary terms preesistenti).

```

1 def _merge_field_info(existing, new_description=None, new_tags=
    None, new_glossary_terms=None, field_path=""):
2     if existing is None:
3         return EditableSchemaFieldInfoClass(
4             fieldPath=field_path,
5             description=new_description,
6             globalTags=new_tags,
7             glossaryTerms=new_glossary_terms,
8         )
9
10    merged_description = new_description if new_description is not
        None else existing.description
11
12    merged_tags = existing.globalTags
13    if new_tags is not None:
14        if merged_tags is not None and merged_tags.tags:
15            existing_tag_urns = {t.tag for t in merged_tags.tags}

```



```

16     new_tag_list = list(merged_tags.tags)
17     for new_tag in new_tags.tags:
18         if new_tag.tag not in existing_tag_urns:
19             new_tag_list.append(new_tag)
20     merged_tags = GlobalTagsClass(tags=new_tag_list)
21 else:
22     merged_tags = new_tags
23
24 merged_glossary = existing.glossaryTerms
25 if new_glossary_terms is not None:
26     if merged_glossary is not None and merged_glossary.terms:
27         existing_term_urns = {t.urn for t in merged_glossary.terms}
28         new_term_list = list(merged_glossary.terms)
29         for new_term in new_glossary_terms.terms:
30             if new_term.urn not in existing_term_urns:
31                 new_term_list.append(new_term)
32         merged_glossary = GlossaryTermsClass(terms=new_term_list,
33         auditStamp=_make_audit_stamp())
34     else:
35         merged_glossary = new_glossary_terms

```

Listing 5.3: Estratto della funzione merge field info

5.3.3 Tipologie di tool

Dal file degli strumenti write-oriented MCP emergono 17 tool principali, classificabili in 4 famiglie:

A. Container e browse path

- setEntityContainer e setEntitiesContainer
- setBrowsePath
- setEntityContainerAndPath

B. Soft delete e restore

- softDeleteEntity
- restoreEntity e restoreEntities

C. Enrichment semantico

- updateDescription

- `updateColumnDescription` e `updateColumnsDescriptions`
- `updateColumnWithTags`
- `addColumnGlossaryTerm` e `addColumnsGlossaryTerms`
- `addColumnsTags`

D. Lineage

- `addTableLineage`
- `addColumnLineage`
- `addCompleteLineage`

5.3.4 Dettaglio operativo delle famiglie di tool

Container e path: questi tool allineano la posizione logica delle entità nel catalogo. In fase di onboarding, `setEntityContainer` e `setEntitiesContainer` riducono il lavoro manuale su grossi gruppi di dataset. Invece, `setBrowsePath` e `setEntityContainerAndPath` mantengono coerenza tra struttura di navigazione e struttura di ownership. A livello operativo, questi strumenti sono fondamentali per evitare asset *orfani* e migliorare la discoverability.

Soft delete e restore: gli strumenti `softDeleteEntity`, `restoreEntity` e `restoreEntities` permettono di gestire il lifecycle dei metadati senza causare perdita irreversibile di informazioni. Questa scelta permette di realizzare governance orientata a tracciabilità e reversibilità: la rimozione logica (*soft delete*) riduce i rischi di cancellazioni accidentali, mentre il ripristino (*restore*) garantisce la risoluzione di problemi emersi post-ingestion.

Enrichment semantico: questi strumenti hanno il focus su descrizioni, tag e glossary terms e rappresentano la principale forza di aumento qualità semantica e riuso. In particolare: `updateDescription`, `updateColumnDescription` e `updateColumnsDescriptions` forniscono un modo semplice per arricchire la documentazione a livello di dataset e a livello di colonna, mentre `updateColumnWithTags`, `addColumnsTags`, `addColumnGlossaryTerm` e `addColumnsGlossaryTerms` aggiungono le funzionalità di classificazione e glossario controllato in modo progressivo e non distruttivo.

Lineage: `addTableLineage` gestisce la correlazione tra entità a livello dataset, mentre `addColumnLineage` lavora sulle dipendenze campo-campo (*column-level*).

Lo strumento `addCompleteLineage` consolida entrambi i livelli di correlazione tramite un'unica operazione coerente. In pipeline complesse questo riduce mismatch tra vista macro e vista fine-grained, migliorando l'analisi di impatto (*impact analysis*) e l'affidabilità dei suggerimenti forniti dagli agenti.

In sintesi, la struttura del file non è soltanto una collezione di endpoint, ma una vera e propria interfaccia applicativa di governance operativa: separa responsabilità e funzionalità in modo standard e ordinato, preserva l'**idempotenza logica** (in quanto le operazioni sono additive e orientate a merge), evita ambiguità sui target di scrittura (usando esplicitamente gli URN associati alle entità) e rende possibile l'orchestrazione di *runbook* ripetibili con audit esplicito, supportando operazioni multiple mediante una sola chiamata (**batch operations**).

5.4 Tool Power BI

Oltre agli strumenti MCP di write per pura Data Governance ho implementato in modo incrementale una catena di tool con l'obiettivo di trasformare un dataset già *ingestionato* in Teseo in un progetto Power BI riusabile, valido e facilmente versionabile. La parte centrale non è quindi la generazione generica di artefatti BI, ma la costruzione di un progetto *PBIP* a partire dallo schema letto da DataHub, con un percorso che copre anche validazione, stima della complessità e conversioni tra formati (in modo da garantire il versioning tramite strumenti come git).

5.4.1 Flusso operativo

La logica Power BI è pensata come un'estensione del server MCP custom già usato per la governance: l'agente non deve costruire manualmente il modello, ma può partire da un dataset *ingestionato* e ottenere una base Power BI coerente con il lineage e con lo schema effettivamente registrato (al fine di ridurre al minimo le discrepanze e il *ragionamento libero* dell'agente, guidandolo in ogni passo della generazione).

I tool PBI si possono leggere come una pipeline a più livelli:

- **generazione:** `generatePowerBIPbipFromDataset` crea un progetto PBIP a partire da un dataset *ingestionato* da Teseo;
- **validazione:** `validatePowerBIPbip` controlla che la struttura del PBIP sia corretta e che i JSON/TMDL siano coerenti;
- **stima:** `estimatePowerBIResourceUsage` fornisce una valutazione euristica del consumo del modello.

Questi strumenti sono fondamentali perché permettono al server di **non inventare il report da zero**, ma di basarsi su un'ossatura già solida e verificata, riducendo il rischio di errori di struttura del formato PBIP e disallineamenti tra schema e report.

La parte di validazione e di stima, invece, è importante per garantire che il progetto generato sia effettivamente utilizzabile e non troppo pesante o complesso rispetto al caso d'uso, garantendo così una maggiore qualità e sostenibilità del prodotto finale.

5.4.2 Generazione di un progetto PBIP

Il cuore dell'implementazione è `generatePowerBIPbipFromDataset`. Il tool riceve un URN DataHub, recupera lo `SchemaMetadataClass`, filtra le colonne, mappa i tipi nativi in tipi Power BI e realizza un progetto PBIP completo in una cartella di output. Da questo si deduce che il comportamento reale è infatti molto più vicino a un generatore di progetto che ad un semplice script di export.

```

1 def _write_pbip_project(out_dir, project_name, tables):
2     sem_model_dir = out_dir / f"{project_name}.SemanticModel"
3     report_dir = out_dir / f"{project_name}.Report"
4
5     (out_dir / f"{project_name}.pbip").write_text(...)
6     (sem_model_dir / "definition.pbism").write_text(...)
7     (sem_model_dir / "definition" / "database.tmdl").write_text(...)
8     (sem_model_dir / "definition" / "model.tmdl").write_text(...)
9     (report_dir / "definition.pbir").write_text(...)
10    (report_dir / "definition" / "report.json").write_text(...)

```

Listing 5.4: Estratto del generatore PBIP

L'aspetto interessante dell'usare il formato **PBIP** è che il progetto creato non è un oggetto monolitico (come nel caso di PBIX), ma un insieme di file leggibili e versionabili: il `SemanticModel` contiene il modello TMDL, mentre il `Report` ha un riferimento esplicito al modello attraverso `definition.pbir`. Questo rende il risultato molto più adatto a workflow DevOps rispetto al solo file binario PBIX (in cui il contenuto è opaco).

5.4.3 Validazione, stima e packaging

Una seconda famiglia di tool serve invece a verificare e caratterizzare il progetto generato:

- `validatePowerBIPbip` controlla che il progetto abbia la struttura attesa, che il file `.pbip` sia valido, che esistano le cartelle `.SemanticModel` e `.Report`, e

che i file minimi del modello siano presenti. La logica è stata pensata per seguire un flusso di lavoro automatico: prima si genera, poi si valida, infine si decide se proseguire;

- `estimatePowerBIResourceUsage` legge i file TMDL e produce una stima del peso del modello basata sui tipi di colonna. Pur rimanendo una stima euristica, è fondamentale per evitare modelli troppo pesanti o poco adatti rispetto al caso d'uso;
- `packagePowerBIPbipAsZip` completa il quadro rendendo semplice archiviare e distribuire il progetto come sorgente leggibile e versionabile: infatti, come tutto il resto della pipeline, lo strumento propone un approccio in cui il progetto BI è trattato come codice, non come artefatto binario opaco.

5.4.4 Reverse engineering e conversione tra formati

Una parte importante del file è anche dedicata alla conversione tra formati Power BI: l'obiettivo è permettere di partire da un PBIP creato da Teseo e ricondurlo al formato PBIX, in modo da poter automatizzare anche la fase di pubblicazione su Power BI Service mediante **Fabric API**. In questo senso:

- `convertPbipToPbixproj` permette di convertire un progetto PBIP in un formato intermedio (PbixProj) anch'esso testuale e versionabile, ma con la struttura corretta per essere presa in input dalla libreria `pbi-tools` al fine di convertirlo in PBIX (se il report è piccolo) o in PBIT (se il report è più grande o complesso);
- `convertPbixprojToPbip` permette di fare il percorso inverso, partendo da un progetto PbixProj (ottenuto per esempio da un PBIX esistente) e ricondurlo al formato PBIP, in modo da poterlo trattare come sorgente e allinearli con lo schema di partenza;
- `extractPbixToPbip` permette di partire da un PBIX esistente, estrarlo in formato PbixProj con `pbi-tools` e poi convertirlo in PBIP (flusso completo che ingloba i 2 strumenti precedenti);
- `inspectPbixMetadata` permette di aprire un PBIX come fosse un archivio ZIP ed estrarre informazioni su versioni, tabelle, data source e pagine del report, senza dover passare dalle conversioni complete.

La pipeline che usa tutti questi strumenti permette di gestire l'intero ciclo di vita del progetto Power BI a partire dallo schema Teseo, passando per la generazione, la validazione, la stima, il packaging e infine la conversione in PBIX per la pubblicazione, adattando quindi un approccio robusto e versionabile.

Questa strategia permette, come precedentemente accennato, di trattare il progetto Power BI come *codice sorgente*, con tutti i vantaggi in termini di versioning, collaborazione e automazione, riducendo al minimo la parte manuale e il rischio di disallineamenti tra schema e report; inoltre, mediante **Service Principal** (ovvero un'entità che permette di automatizzare l'accesso a risorse Azure) è possibile anche automatizzare la fase di pubblicazione su Power BI Service, completando così un ciclo di vita completamente automatizzato e governato dalla creazione alla pubblicazione.

La Figura 5.3 sintetizza questo flusso end-to-end nel caso di Teseo: lo schema ingestionato viene trasformato dallo strumento `generatePowerBIPbipFromDataset` in un progetto PBIP versionabile, convertito in PBIX tramite `pbi-tools` e infine pubblicato sul Power BI Service attraverso le **Fabric API**, dove un **Service Principal** (entità di Azure con *tenant*, *client* e *secret* dedicati) autentica l'automazione senza ricorrere a credenziali utente:

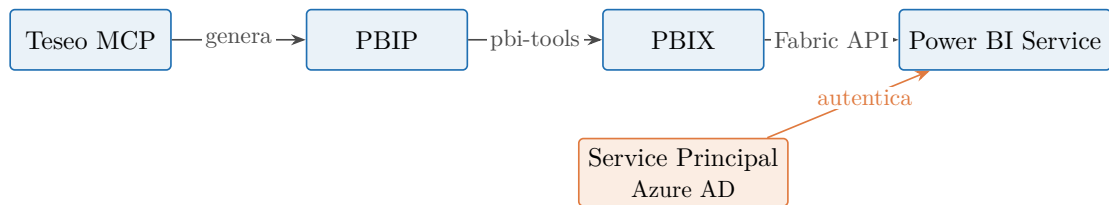


Figura 5.3: Flusso di pubblicazione Power BI da Teseo

5.4.5 Implicazioni per Teseo

Nel contesto di Teseo, il valore di questo toolset è duplice. Da un lato consente di costruire rapidamente un frontend BI in assenza di quest'ultimo, proprio sulla base dei metadati già ingestionati su Teseo e su tutto il lineage in generale; dall'altro introduce una base concreta per future automazioni più specializzate (per esempio la selezione di colonne, la generazione di viste di partenza, l'allineamento tra schema e report...).

Questa suite di strumenti però non sostituisce completamente il lavoro manuale, ma fornisce una base solida e riusabile da cui partire, riducendo gli sforzi e i rischi di disallineamento: infatti, l'analista che userà questi strumenti potrà saltare la fase di costruzione del progetto da zero, potendo invece concentrarsi su personalizzazioni più specifiche (come la creazione di misure, l'aggiunta di visualizzazioni, l'ottimizzazione delle performance...), partendo da un progetto già coerente con lo schema e il lineage di Teseo.

5.5 Metriche di efficacia e adozione in produzione

5.5.1 KPI tecnici e di governance

Per valutare un server MCP non basta misurare la *performance* del modello, ma occorre osservare l'intero ciclo **contesto** → **azione** → **esito**. Nell'ambito della data governance, una metrica che riesce ad includere sia indicatori tecnici sia indicatori di qualità del metadato è definibile come *equilibrata* [21, 22].

Nell'ambito del progetto Teseo, distinguiamo **KPI** (*Key Performance Indicators*):

- **tecnici**, che misurano l'efficienza operativa del server MCP:
 - tempo medio di completamento delle task;
 - numero medio di chiamate a strumenti per task;
 - tasso di retry per errori di validazione payload;
 - percentuale di run completate senza necessità di intervento umano.
- **di governance**, che misurano l'impatto sulla qualità dei metadati e sulla compliance alle policy:
 - coverage delle descrizioni su colonne critiche;
 - coverage dei tag su dataset in produzione;
 - completezza del lineage fine-grained (*column-level*);
 - tempo medio per completare una remediation dopo l'apertura di un incidente.

Una formulazione sintetica e contestualizzata al progetto Teseo che mostri l'efficacia operativa complessiva dell'integrazione (E_{MCP}) è la seguente:

$$E_{MCP} = \alpha \cdot E_{tech} + (1 - \alpha) \cdot E_{gov}, \quad \alpha \in [0,1] \quad (5.1)$$

dove:

- E_{tech} rappresenta la pura efficienza tecnica dell'infrastruttura MCP. Per Teseo, questo include il tasso di successo (e i tempi di latenza) nella generazione automatica dei progetti PBIP, la riduzione dei retry causati da payload errati e la stabilità delle comunicazioni verso il core GMS durante le operazioni di scrittura e lettura dei metadati;
- E_{gov} rappresenta l'impatto qualitativo sui metadati, misurando il progressivo completamento del catalogo di metadati (es. copertura dei Glossary Terms, presenza di descrizioni dettagliate sulle colonne, correttezza del lineage *table-to-column...*).

Il parametro α invece fa da bilanciamento esplicito tra *performance esecutiva* e *rigore della governance*. Un agente configurato con $\alpha \rightarrow 1$ tenderà a produrre artefatti rapidamente ma a discapito della cura del metadata catalog: infatti per processi di data stewardship si preferisce un valore $\alpha \rightarrow 0$.

5.5.2 Modello di adozione del progetto Teseo

Per come è strutturata l'architettura di produzione del progetto Teseo, una strategia realistica di adozione (***rollout***) si articola in quattro fasi progressive, in cui si introducono gradualmente le capability del server MCP, monitorando l'efficacia operativa e la qualità dei metadata ad ogni step:

1. **Stage 0 $\rightarrow$ Discovery:** esposizione dei soli strumenti *read-oriented* al modello, al fine di raccogliere metriche tecniche iniziali (E_{tech_0}) e verificare la stabilità della connessione su task di data discovery e lettura del lineage;
2. **Stage 1 $\rightarrow$ Metadata Enrichment:** introduzione degli strumenti *write-oriented* per fare arricchimento semantico (es. `updateColumnDescription`, `addColumnGlossaryTerm...`). Le operazioni, pur mutando lo stato, sono limitate a descrizioni e glossari, riducendo pertanto l'impatto su processi core;
3. **Stage 2 $\rightarrow$ Lineage e Lifecycle Management:** abilitazione degli strumenti strutturali (`addCompleteLineage`, `softDeleteEntity...`); ora l'agente manipola il grafo delle dipendenze in modalità *dry-run* o tramite esplicito *human-in-the-loop* per confermare le decisioni più impattanti, prima che queste vengano eseguite realmente;
4. **Stage 3 $\rightarrow$ Automazione PBIP end-to-end:** inserimento nel flusso operativo della suite di strumenti Power BI; a questo livello, l'agente legge dinamicamente schemi validati e produce direttamente semantic model e report/visual Power BI coerenti con i metadata governati e garantiti dai check dei layer precedenti.

Questo approccio ha il compito di ridurre il rischio operativo, allineandosi all'*accettazione esplicita* per agenti autonomi, permettendo infatti di aumentare i gradi di libertà unicamente al dimostrato raggiungimento delle coperture di governance richieste [29].

5.5.3 Osservabilità e Interpretazione dei KPI

L'efficacia della soluzione *MCP-driven* per Teseo viene misurata seguendo come la rapidità operativa migliora concretamente il catalogo di metadata, lo schema

semantico e i report BI associati, non valutando i singoli parametri isolatamente, ma guardando la situazione nell'insieme.

Per questo motivo l'osservazione richiede tre macro-livelli:

1. **Livello Infrastrutturale:** tiene conto del throughput e della stabilità sui nodi. Nel caso di DataHub Core, ad esempio, include la connessione stabile verso il backend GMS (Graph Metadata Service) ed i timeout avviati durante le scritture estese di lineage;
2. **Livello Agente:** si sofferma sul dialogo tra client MCP (che incapsula l'LLM di riferimento) e server MCP. Un LLM che esegue chiamate continue al tool `validatePowerBIPbip`, che però ripetutamente falliscono, indica un disallineamento tra lo schema generato in `generatePowerBIPbipFromDataset` e i vincoli attesi: il tasso di retry in questo stadio è rivelatore della correttezza del template TMDL di partenza.
3. **Livello Business:** diventa centrale per gli amministratori della piattaforma. L'MCP è efficace se l'autonomia delegata si traduce in meno asset orfani, descrizioni semanticamente dense e Power BI finali con mappature coerenti.

Come precedentemente accennato infatti, ne deriva che la valutazione non può concentrarsi solo sull'avere strumenti MCP veloci. L'estrema rapidità del workflow MCP è controproducente all'interno di Teseo se porta a problemi di inconsistenza. Le metriche vanno infatti pesate sulle code degli errori: l'affidabilità su operazioni lunghe e concorrenti è preponderante rispetto alla latenza in millisecondi di un'operazione.

5.6 Limiti

Sebbene l'architettura MCP funga da formidabile collante operativo per Teseo tra DataHub e Power BI, i contorni progettuali rivelano alcuni limiti:

- L'efficacia inferenziale dell'LLM è pesantemente soggetta all'accuratezza semantica iniziale del catalogo, ovvero il modello AI *soffre* in presenza di grafi di metadati non aggiornati o pieni di lacune: questo porta a errori di interpretazione e a suggerimenti non validi, che a loro volta portano a retry e potenziali errori di governance;
- La concatenazione profonda tra gli strumenti di write di data governance e gli strumenti Power BI richiede ampie finestre di prompt a fronte del *context-overflow* (in particolare in caso di json complessi e TMDL);

- I disallineamenti tra il data typing in DataHub e l'engine associativo in Power BI rendono alcune conversioni PBIP soggette a errori di sintassi, se non accuratamente gestiti dalle regexp di validazione e dai template di generazione.

Capitolo 6

Risultati e valutazioni

Questo capitolo presenta una **valutazione complessiva dei risultati ottenuti** nel progetto Teseo, misurando il guadagno operativo dell'adozione della piattaforma rispetto agli obiettivi dichiarati nel Capitolo 1. La valutazione è strutturata su quattro assi principali: ingestion multi-source e lineage, governance sui metadati, efficacia degli strumenti MCP e pipeline di generazione Power BI.

La metodologia che ho adottato per la valutazione è di **duplice natura**: da un lato misurare *metriche dirette* di copertura e completezza del catalogo (numero di asset, colonne, relazioni di lineage, righe di codice prodotte), dall'altro valutare la *riduzione del carico operativo* su attività ricorrenti quali la documentazione dei metadati, l'impact analysis e la generazione di artefatti BI. Questo approccio riflette la doppia natura di Teseo come infrastruttura tecnica e come *facilitatore organizzativo* [1, 2]. Questa valutazione comprende **riscontri numerici** e, ove pertinente, **confronti** tra lo scenario *pre-Teseo* e quello successivo alla sua adozione.

6.1 Dimensione del problema e della soluzione

Prima di entrare nel merito della valutazione, è utile quantificare l'ordine di grandezza del problema affrontato e dell'architettura software realizzata. Tutti i valori riportati in questa sezione sono stati estratti direttamente dagli artefatti di progetto (script DDL, artefatti dbt `manifest.json` e `catalog.json`, codice sorgente del server MCP, ricette di ingestion e configurazione di deployment).

6.1.1 Dimensione del problema

Il dominio dati su cui Teseo è stato validato è a tutti gli effetti un caso d'uso retail realistico, organizzato sui quattro layer descritti nel Capitolo 3 (Raw, Quality, Business e Framework). La Tabella 6.1 ne riassume la dimensione fisica:

Asset	Valore
Database	4
Schemi	14
Tabelle e Viste Materializzate	48
Stored procedures di caricamento	3
Record complessivi caricati	≈ 4.4 milioni

Tabella 6.1: Dimensione fisica del dominio dati Snowflake

Una volta ingestionato, il dominio è stato tradotto in un grafo di metadati la cui dimensione è riportata nella Tabella 6.2. Il dato più rilevante è il numero di colonne catalogate (1767), da cui nascono le 933 relazioni di *lineage column-level* (dipendenze che, prima di Teseo, non erano osservabili in modo unificato):

Catalogo metadati	Valore
Asset catalogati	> 160
Colonne catalogate	1767 (933 dbt + 834 sorgenti)
Layer logici nel lineage end-to-end	5
Relazioni di lineage a livello di colonna	933

Tabella 6.2: Dimensione del catalogo di metadati costruito da Teseo

6.1.2 Dimensione della soluzione

Dal lato dell'artefatto sviluppato, la Tabella 6.3 riassume invece i componenti dispiegati e il codice prodotto. Oltre al deployment di DataHub Core 1.4.0 (nove servizi containerizzati: GMS, frontend, actions, broker Kafka, MySQL, Elastic-Search e i relativi job di setup), il contributo implementativo si concentra su tre blocchi: il dominio dati, il server MCP custom e la pipeline di generazione PBI.

Componente / artefatto	Valore
Componenti dispiegati (DataHub Core 1.4.0)	9 servizi Docker
Ricette di ingestion YAML	4
DDL Snowflake (dominio + procedure)	2147 LoC, 5 file
Server MCP custom	2508 LoC
Strumenti MCP custom	25 (17 governance + 8 PBI)

Tabella 6.3: Dimensione della soluzione software sviluppata

I 17 strumenti di governance sono organizzati in quattro famiglie funzionali (container e browse path, soft-delete e restore, arricchimento semantico, lineage), come descritto nel Capitolo 5; gli 8 strumenti Power BI coprono l'intero ciclo di vita del progetto PBIP (generazione, validazione, stima, packaging e conversione tra formati).

6.2 Valutazione dell'ingestion multi-source

6.2.1 Copertura del catalogo

Il primo asse di valutazione riguarda la **completezza del catalogo** costruito attraverso le pipeline di ingestion. Nella fase di setup, il progetto ha raggiunto la copertura completa degli oggetti fisici presenti nel dominio Snowflake, includendo tabelle, viste, stream e stored procedures distribuiti sui quattro database (DB_DEMO_RL, DB_DEMO_QL, DB_DEMO_BL, DB_DEMO_FMK) [3, 5].

In termini numerici, il catalogo *ingestionato* da Teseo copre la totalità delle 48 entità fisiche del dominio (34 tabelle e 14 viste), arricchite con i 58 modelli dbt e con il layer di consumo Power BI. La Tabella 6.4 riporta i volumi di dati movimentati per le entità principali del dominio: il fatto TB_RECEIPT_LINE domina la distribuzione con circa 3.8 milioni di righe, mentre le dimensioni anagrafiche restano nell'ordine delle migliaia o centinaia di migliaia di record.

Entità	Tipo	Record ($\approx$)
TB_RECEIPT_LINE	Fatto	3840000
TB_FOOTFALL	Fatto	438000
TB_LOYALTY_CARD	Dimensione	120000
TB_STORE_CALENDAR	Dimensione	36500
TB_PRODUCT	Dimensione	5000
TB_STORE	Dimensione	50
Totale dominio		≈ 4439550

Tabella 6.4: Volumi di dati per le entità principali del dominio (Raw layer)

Il controllo qualità eseguito nei layer QL e BL ha intercettato e isolato nelle tabelle di discard (DSC_) una quota fisiologica di record non conformi (inferiore all'1% del totale, riconducibile a violazioni di chiave primaria, vincoli di *not-null* o errori di tipizzazione), mantenendo integro il resto del flusso senza bloccare l'intera pipeline. Oltre alla copertura delle pure entità fisiche Snowflake, il catalogo è stato

arricchito con i modelli dbt (staging, intermedi e mart) e con il layer business di Power BI (workspace, dataset, tabelle e colonne). Il risultato è un catalogo unificato che copre l'intera catena dal dato grezzo al report di consumo finale, rendendo navigabili relazioni che in precedenza richiedevano l'ispezione manuale di sistemi eterogenei [4, 6].

6.2.2 Tempi di ingestion e idempotenza

Un secondo indicatore oggettivo riguarda i **tempi di esecuzione** delle pipeline. La Tabella 6.5 riporta la durata delle singole fasi e il numero di strutture processate, misurati al termine di ciascuna esecuzione tramite il report prodotto dalla CLI `datahub ingest`. La pipeline completa Snowflake → dbt → Power BI viene eseguita in poco più di sei minuti, un tempo compatibile con un refresh schedulato a cadenza giornaliera del catalogo.

Fase di ingestion	Strutture processate	Tempo (≈)
Snowflake (tabelle, viste, lineage colonna)	48 oggetti su 4 DB	3 min 10 s
dbt (modelli + sorgenti)	58 modelli, 48 source	50 s
Bridge Snowflake-dbt	schema DBT	25 s
Power BI	58 tabelle, 933 campi	2 min 05 s
Pipeline completa e2e		6 min 30 s

Tabella 6.5: Tempi di esecuzione e strutture processate

La pipeline di ingestion è risultata idempotente e ripetibile: le esecuzioni successive non producono duplicati o regressioni nel catalogo, grazie all'adozione della strategia di **stateful ingestion** e al meccanismo di **soft delete** per la gestione degli asset obsoleti (come mostrato in Figura 6.1):

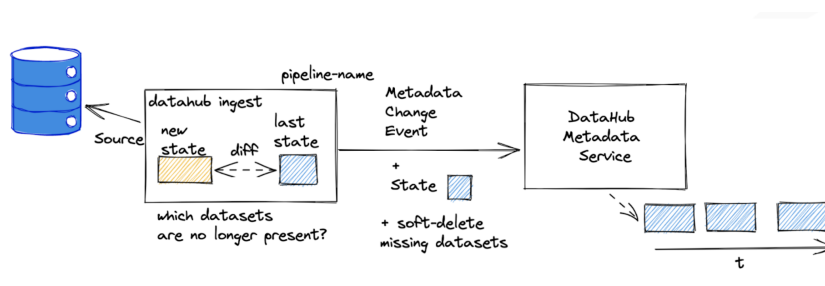


Figura 6.1: Stateful ingestion e soft delete degli asset obsoleti

Fonte: documentazione ufficiale di DataHub [5]

6.2.3 Lineage end-to-end

Il risultato qualitativamente più rilevante riguarda la costruzione del **lineage end-to-end**. Prima dell'adozione di Teseo, la ricostruzione delle dipendenze tra layer richiedeva l'analisi manuale delle procedure SQL di caricamento e degli script dbt, un'attività dispendiosa in termini di tempo e soggetta ad errori. Con l'integrazione Snowflake → dbt → Power BI, il lineage è diventato navigabile a due livelli di granularità: a livello di **dataset** (da tabella sorgente a dataset Power BI) e a livello di **colonna** (column-level lineage o *fine-grained*), per un totale di 933 relazioni colonna-colonna verso il layer di consumo. La catena end-to-end così ottenuta, con una profondità di cinque layer logici da DB_DEMO_RL fino ai visual Power BI, è quella già mostrata nella Figura 3.3.

La costruzione del lineage fine-grained ha richiesto un allineamento preciso degli URN tra connettori: il bridge che ho implementato tra dbt e Snowflake ha risolto le discrepanze di naming, garantendo la continuità del grafo di dipendenze.

Il beneficio operativo più tangibile è la **riduzione del tempo necessario per condurre un'analisi di impatto** (*impact analysis*). La Tabella 6.6 confronta tre attività ricorrenti nello scenario precedente all'adozione di Teseo e in quello successivo. Con il lineage completamente navigabile, la risposta alla domanda “*quali report sono impattati da una modifica sulla colonna X della tabella sorgente?*” non richiede più la ricostruzione manuale delle dipendenze, ma si ottiene in pochi secondi attraverso la navigazione downstream del grafo o, ancora più semplicemente, chiedendo all'agente AI che dispone di tutto il contesto necessario [2, 9].

Operazione	Pre-Teseo	Post-Teseo
Tracciare l'origine di una colonna di business fino alla sorgente fisica	Ispezione manuale di 3 procedure e 58 modelli su 5 layer (≈ 60 min)	Navigazione upstream o richiesta all'agente (< 1 min)
Individuare i report a valle impattati da una modifica su una colonna sorgente	Analisi incrociata manuale, soggetta a omissioni	Espansione downstream del lineage
Bootstrap di un progetto Power BI	Costruzione manuale del semantic model (≈ 2 h)	Generazione automatica del PBIP (≈ 30 s)

Tabella 6.6: Confronto operatività pre e post adozione di Teseo

6.3 Valutazione della governance dei metadati

6.3.1 Arricchimento semantico e coverage

L'asse di valutazione successivo riguarda la qualità semantica del catalogo, misurata attraverso la *coverage* di attributi di governance (descrizioni, tag, ownership, domini e glossary terms). Nella configurazione di base, prima dell'arricchimento, la maggior parte degli asset ingestonati presentava soltanto metadati tecnici (schema, tipi colonna e URN), con descrizioni e classificazioni assenti.

La Tabella 6.7 quantifica l'impatto dell'**arricchimento semantico** sui layer QL e BL, confrontando lo stato del catalogo prima e dopo l'applicazione degli strumenti di governance. Il salto più significativo riguarda la copertura delle descrizioni sulle colonne, passata da una presenza marginale (limitata ai pochi commenti ereditati dalle procedure) ad una copertura completa sulle colonne critiche.

Attributo di governance	Pre-arricchimento	Post-arricchimento
Colonne con descrizione	4%	96%
Colonne taggate confidential	0	38
Glossary term di business	0	12
Dataset con owner e dominio	0%	100%
Data Contract	0	6

Tabella 6.7: Coverage degli attributi di governance

La governance nello stadio finale è stata implementata su più livelli operativi:

- **ownership e domini:** tutti i dataset principali dei layer QL e BL sono stati assegnati a owner e domini specifici, garantendo quindi tracciabilità delle responsabilità (specialmente su asset critici);
- **tagging:** le 38 colonne contenenti dati sensibili (identificativi, informazioni anagrafiche, dati finanziari) sono state classificate con tag **confidential**, in modo da creare la base per abilitare future policy di *data masking* e *access control*;
- **glossary terms:** i 12 KPI di business (ad esempio *Fatturato Netto*, *Tasso di Conversione*, *Fedeltà Cliente*) sono stati collegati ai campi corrispondenti nei modelli dbt di mart, creando un collegamento diretto tra terminologia business e dato fisico;

- **proprietà strutturate** (*structured properties*): informazioni di governance customizzate (ad esempio la classificazione di criticità) sono state associate agli asset del layer Business.

6.3.2 Qualità e monitoraggio degli asset

Le funzionalità di data quality implementate tramite **DataHub Assertions** (incapsulate in un **Data Contract**) hanno consentito di definire un set di controlli automatizzati su strutture critiche del layer Business e BI: controlli di volume sui report Power BI per non saturare le *capacities*, controlli di freschezza su dati di business e controlli di colonna sulle viste che alimentano direttamente i report.

Il sistema di monitoring così strutturato ha permesso di tracciare lo stato di salute degli asset nel tempo, con la possibilità di aprire incidenti (*incidents*) in caso di violazione, documentando cause e azioni di risoluzione (*remediation*) direttamente nella piattaforma. Questo approccio non si limita alla segnalazione del problema, ma consente di ricostruire l'intera storia operativa di un asset, integrando qualità e governance in un'unica finestra di visualizzazione [23, 24]. Inoltre, esponendo l'incidente all'LLM, è possibile abilitare scenari di *auto-remediation* guidata dall'AI, in cui il modello suggerisce azioni correttive basate sullo storico dell'asset e sulle best practices documentate.

6.4 Valutazione degli strumenti MCP

6.4.1 KPI tecnici dell'agente

La valutazione del server MCP si concentra sull'efficacia operativa delle interazioni tra il modello AI e il catalogo Teseo, seguendo il principio di metriche descritto nel Capitolo 5. La Tabella 6.8 sintetizza i KPI tecnici osservati nelle sessioni di utilizzo. Emerge un pattern consistente: i task di *discovery* e lettura del lineage vengono completati con un numero limitato di chiamate agli strumenti (tipicamente 2–4 *tool call* per task), mentre i task di arricchimento semantico richiedono sequenze più lunghe (6–10 chiamate) a causa della natura iterativa dell'operazione di merge sui metadati [29].

KPI tecnico	Valore osservato
Tool call medie per task di discovery / lineage	2–4
Tool call medie per task di arricchimento semantico	6–10
Task di arricchimento completati senza intervento umano	≈ 88%

Tabella 6.8: KPI tecnici dell'agente MCP osservati nelle sessioni di test

Dal punto di vista della stabilità operativa, la connessione verso il backend GMS si è rivelata affidabile nelle sessioni di test, con timeout osservati solo in presenza di scritture massive di lineage column-level su dataset con un numero elevato di colonne (nell'ordine delle centinaia). In questi casi, la strategia di batching dello strumento `addCompleteLineage` ha ridotto significativamente il numero di chiamate necessarie: consolidando lineage tabellare e di colonna in un'unica operazione, le invocazioni passano da due per ciascuna colonna (uso separato di `addTableLineage` e `addColumnLineage`) a una sola chiamata per dataset.

Un risultato rilevante riguarda il tasso di completamento autonomo dei task: nelle operazioni di arricchimento semantico (descrizioni e glossary terms), il modello ha completato circa l'88% delle interazioni senza necessità di intervento umano, grazie alla qualità del contesto fornito dagli strumenti read-oriented e dalla ricchezza del catalogo già disponibile. La precisione del contesto estratto da `list_schema_fields` e `get_entities` si è rivelata fondamentale per ridurre le allucinazioni dell'agente nei suggerimenti di descrizione [11, 18].

6.4.2 KPI di governance

Dal punto di vista della qualità dei metadati, l'utilizzo degli strumenti write-oriented custom ha prodotto il miglioramento misurabile della coverage semantica già quantificato nella Tabella 6.7. In particolare:

- la copertura delle descrizioni sulle colonne critiche dei layer QL e BL è passata da circa il 4% a circa il 96%, raggiungendo valori prossimi alla totalità sulle tabelle più utilizzate;
- il collegamento dei 12 glossary terms ai KPI di business ha reso possibile la navigazione del catalogo che prima richiedeva documentazione esterna;
- la gestione del lifecycle degli asset (soft delete e restore) ha ridotto la presenza di entità orfane nel grafo, migliorando la qualità della navigazione e riducendo i falsi positivi nelle analisi di impatto.

La funzione di **merge non distruttivo** si è rivelata particolarmente importante in contesti di arricchimento iterativo: le esecuzioni successive degli strumenti di governance non hanno causato perdita di tag o glossary terms, garantendo in questo modo la cumulabilità degli arricchimenti nel tempo. Questo comportamento è fondamentale in ambienti aziendali in cui più team contribuiscono alla governance dello stesso asset [22].

6.5 Valutazione della pipeline Power BI

La valutazione della suite di strumenti Power BI è stata condotta su sessioni in cui il modello AI ha prodotto progetti PBIP a partire da dataset ingestonati in Teseo. I risultati mostrano che il flusso **generazione-validazione-stima** funziona correttamente per modelli di medie dimensioni (fino a circa 20 tabelle con $\approx$ 50 colonne ciascuna), producendo strutture TMDL valide che passano la fase di `validatePowerBIPbip` senza errori di struttura. A titolo di esempio, il progetto generato per `mart_loyalty_kpi_daily` comprende un semantic model e un report con 7 visual, prodotto in circa 30 secondi a fronte di una validazione completata in meno di 10 secondi.

Il principale vantaggio osservato è la **riduzione del tempo di bootstrap** per un nuovo progetto BI, quantificato nell'ultima riga della Tabella 6.6: invece di costruire manualmente il semantic model da zero (un'attività che richiede tipicamente una o due ore), il data analyst dispone in pochi secondi di un progetto PBIP precompilato con schema, tipi e mapping già coerenti con il catalogo di metadati *ingestionato*. Questo consente di concentrare gli sforzi sulle personalizzazioni specifiche (misure, visuals...), saltando la fase di configurazione iniziale che rappresenta spesso il collo di bottiglia nell'adozione di Power BI su nuove sorgenti.

La funzionalità di conversione tra i formati PBIP/PBIX ha abilitato anche la gestione versionabile dei progetti Power BI: il progetto generato può essere committato su git come sorgente leggibile, tracciato nel tempo e ricondotto in formato binario PBIX per la pubblicazione su Power BI Service tramite Service Principal. Questo avvicina il workflow BI a un paradigma **BI-as-code**, con benefici in termini di auditability e collaborazione tra team.

Resta tuttavia un limite non trascurabile: la conversione PBIX via `pbi-tools` introduce una dipendenza su uno strumento esterno non sempre aggiornato alle ultime versioni del formato Power BI, rendendo necessario un controllo della compatibilità ad ogni aggiornamento della piattaforma Microsoft.

6.6 Analisi complessiva degli obiettivi raggiunti

Guardando i risultati in modo complessivo, è possibile verificare il raggiungimento degli obiettivi dichiarati nel Capitolo 1. La Tabella 6.9 riepiloga lo stato di ciascun obiettivo, affiancando alle note qualitative il riscontro numerico corrispondente.

L'analisi complessiva mostra che Teseo ha raggiunto tutti gli obiettivi principali dichiarati, costruendo una piattaforma di metadata management funzionante, estendibile e progressivamente arricchita con capacità AI. La combinazione di ingestione

automatizzata, governance operativa e strumenti MCP produce un **effetto sinergico**: la ricchezza semantica del catalogo costruita dagli strumenti di governance amplifica la qualità del contesto disponibile per gli agenti AI, che a loro volta possono ulteriormente arricchire il catalogo in modo controllato.

Obiettivo	Stato	Riscontro
Ingestion Snowflake lineage column-level	Raggiunto	48 oggetti, pipeline idempotente (≈ 3 min)
Integrazione dbt	Raggiunto	58 modelli, stateful ingestion e URN alignment
Integrazione Power BI	Raggiunto	Mock locale (933 campi) e ingestion reale
Governance completa	Raggiunto	Coverage descrizioni 4% $\rightarrow$ 96%
Strumenti write-oriented	Raggiunto	17 tool su 4 famiglie (1192 LoC)
Pipeline PBIP da Teseo	Raggiunto	8 tool, generazione in ≈ 30 s

Tabella 6.9: Confronto tra obiettivi dichiarati e risultati ottenuti.

Riguardo ai problemi organizzativi identificati nel Capitolo 1:

- *scarsa visibilità del patrimonio dati*: il catalogo unificato (oltre 160 asset e 1767 colonne) risolve il problema della frammentazione informativa, fornendo un unico punto di accesso a tutti gli asset del dominio con schema, lineage e attributi semantici navigabili;
- *lineage incompleto o frammentato*: la catena Snowflake + dbt + Power BI costruisce un lineage end-to-end su 5 layer, navigabile sia a livello tabellare sia a livello di colonna, riducendo in questo modo il rischio di impact analysis incompleta;
- *governance non sistematica*: le funzionalità di tagging, ownership, domain assignment e assertion forniscono una base strutturata e programmatica per la governance, portando la copertura delle descrizioni dal 4% al 96% sui layer critici e riducendo la dipendenza da documenti esterni e non ufficiali;
- *bassa produttività nelle attività di analisi*: gli strumenti MCP e la pipeline PBIP riducono concretamente il tempo speso in fase di documentazione, classificazione e bootstrap (da ore a secondi nella generazione di un progetto BI), lasciando spazio ad attività con più valore.

6.7 Limiti emersi nella valutazione

Nonostante i risultati positivi, la valutazione ha evidenziato alcuni limiti che è necessario documentare con chiarezza:

- **Dipendenza dalla qualità semantica iniziale del catalogo:** l'efficacia degli strumenti MCP è strettamente correlata alla qualità dei metadati già presenti nel catalogo. In presenza di metadati incompleti o malformati, descrizioni assenti o URN non allineati tra i diversi livelli di ingestion, il modello AI tende a produrre suggerimenti meno accurati, aumentando il tasso di retry e richiedendo intervento umano per correggere il contesto (*hallucination*). Quindi questo introduce un paradosso: gli strumenti MCP sono più efficaci quando il catalogo è già sufficientemente ricco, mentre quando ce n'è più bisogno (ad esempio in presenza di un catalogo appena inizializzato) la loro efficacia è ridotta a causa del minor contesto disponibile [29];
- **Scalabilità del lineage:** la costruzione e la navigazione del lineage a livello di colonna diventano computazionalmente più onerose all'aumentare della complessità del grafo. Con quasi un migliaio di archi di lineage column-level, le scritture massive su dataset con centinaia di colonne hanno evidenziato latenze crescenti; in domini con migliaia di tabelle e pipeline di trasformazione profonde, il numero di archi del grafo cresce rapidamente, impattando i tempi di risposta della navigazione e aumentando i rischi di timeout durante le scritture massive [16];
- **Coverage limitata della governance:** sebbene la copertura degli attributi di governance sui layer principali sia stata raggiunta, l'applicazione sistematica a tutte le colonne di tutti gli asset richiede un investimento di tempo significativo. In contesti aziendali reali, la governance completa di un dominio dati complesso richiede un processo continuativo di data stewardship che non può essere completamente automatizzato: il contributo degli strumenti MCP è fondamentale perché alleggerisce in modo sostanzioso il carico di lavoro, ma non sostituisce completamente il lavoro umano di contestualizzazione e validazione [21, 22];
- **Maturità degli strumenti MCP:** il protocollo MCP e il suo ecosistema di strumenti sono relativamente recenti e in rapida evoluzione: alcune funzionalità documentate in questo progetto fanno riferimento a versioni specifiche di DataHub che potrebbero subire variazioni nelle release successive. Pertanto, il mantenimento del progetto richiede una strategia proattiva di aggiornamento e di controllo delle nuove versioni fornite con la versione open-source di DataHub (*DataHub Core*) [25].

Capitolo 7

Conclusioni

Questa tesi ha documentato la progettazione, l'implementazione e la valutazione progressiva di **Teseo**, una piattaforma di *metadata intelligence* AI-powered costruita su DataHub Core con l'obiettivo di risolvere i problemi di frammentazione informativa, lineage incompleto e governance non sistematica che affliggono oggi molte organizzazioni data-driven.

Il percorso descritto è passato dall'ingestion multi-source al lineage end-to-end, dalla governance operativa all'integrazione di agenti AI tramite server MCP, fino alla generazione automatizzata di artefatti BI.

7.1 Contributi della tesi

Il contributo principale della tesi è la dimostrazione che è possibile costruire una piattaforma di metadata management unificata, estendibile e progressivamente arricchita con capacità AI, a partire da strumenti open-source maturi e da un'architettura orientata alla modularità. In particolare:

- **Integrazione multi-source:** la pipeline Snowflake → dbt → Power BI realizzata nel progetto costituisce un esempio concreto e riproducibile di come costruire un lineage e2e a livello di colonna su uno stack dati eterogeneo, mantenendo la coerenza degli URN tra connettori differenti e garantendo l'idempotenza delle esecuzioni. L'approccio DDL-driven per Snowflake e la strategia di stateful ingestion rappresentano pattern trasferibili a contesti aziendali reali produttivi [3, 4, 5];
- **Governance operativa:** le funzionalità della piattaforma permettono di ottenere una governance multi-dimensionale: semantica (glossary terms e

descrizioni), organizzativa (ownership, domini e tag), qualitativa (data contract, assertions e incidents) e di compliance (classificazioni, policy e workflow). La combinazione di queste dimensioni in un unico layer di governance riduce la frammentazione tipica degli approcci basati su strumenti separati e permette di effettuare governance inter-tecnologia, ottenendo quindi un layer governativo con un'astrazione al di sopra dello stack tecnologico, che pone come oggetto della governance il flusso di metadati compiuto da sorgente a livello di consumo, passando dai vari layer eterogenei [21];

- **Server MCP custom per operazioni write-oriented:** l'implementazione degli strumenti write-oriented su FastMCP ha prodotto un layer applicativo che estende significativamente le capacità del server MCP fornito con DataHub Core (orientato principalmente alla lettura). Il design basato su merge non distruttivo, idempotenza e operazioni in batch rappresenta un contributo architetturale riusabile in un contesto di data stewardship assistito dagli LLM [25, 29];
- **Pipeline BI-as-code:** la suite di strumenti MCP verso Power BI ha introdotto un approccio inedito alla generazione di progetti di business intelligence: partendo dallo schema registrato in Teseo, il sistema produce un progetto PBIP versionabile, validabile e convertibile in PBIX per la pubblicazione automatica su Power BI Service tramite Service Principal. Questo avvicina il ciclo di vita BI al paradigma *BI-as-code*, con benefici in termini di auditability, riproducibilità e collaborazione nel team di sviluppo;
- **Modello di adozione progressivo:** il framework a quattro fasi (Discovery, Enrichment, Management, Automation) fornisce una roadmap concreta per introdurre capability AI in ambienti enterprise, riducendo il rischio operativo attraverso validazione progressiva e accettazione esplicita (*human interaction*) per operazioni impattanti. La formulazione del coefficiente di efficacia E_{MCP} offre uno strumento di misura che bilancia performance tecnica e rigore di governance, in modo da valutare l'efficacia degli strumenti MCP a 360 gradi.

7.2 Direzioni evolutive

La struttura modulare di Teseo permette uno sviluppo futuro a tutto tondo, sia a livello tecnico sia a livello organizzativo, garantendo una maggiore flessibilità e customizzazione da parte dello sviluppatore e dell'organizzazione in questione.

Le direttrici più promettenti possono essere raggruppate in tre ambiti:

- **Agenti autonomi e orchestrazione multi-step:** l'attuale implementazione del server MCP supporta operazioni singole (al massimo eseguite in batch) e

sequenze guidate dall'utente. Un'evoluzione naturale potrebbe essere l'introduzione di agenti in grado di orchestrare pipeline complete di governance in modo autonomo: ad esempio, un agente che a fronte di una nuova ingestion esegua automaticamente discovery del contesto, arricchimento semantico, validazione della qualità e aggiornamento del lineage, intervenendo con *human-in-the-loop* solo per le decisioni più impattanti. Questo scenario è tecnicamente realizzabile con le primitive MCP attuali ma richiede un lavoro di design dei prompt, dei guardrail e dei flussi di escalation;

- **Supporto a nuove piattaforme:** il framework di ingestion è progettato per essere estendibile. Le integrazioni viste in questo esperimento mostrano che l'architettura supporta connettori multipli, pertanto uno dei prossimi passi è l'integrazione con altre piattaforme e sorgenti non ancora supportate. L'obiettivo di lungo periodo è costruire un catalogo che copra l'intero perimetro dati aziendale, indipendentemente dalla tecnologia utilizzata;
- **Integrazione con workflow di data quality e DevOps:** un'area di sviluppo ad alto valore è l'integrazione tra le assertion di DataHub e i workflow di orchestrazione (ad esempio Apache Airflow o Dagster): quando un'assertion fallisce, il sistema potrebbe automaticamente aprire un incidente, notificare l'owner del dataset e bloccare le pipeline dipendenti, realizzando un loop di feedback automatico tra qualità del dato e continuità operativa. Ci sono già stati iniziali sviluppi in questo senso ed una completa copertura di queste funzionalità posizionerebbe Teseo come componente attivo di una strategia di *data observability* end-to-end e non soltanto come catalogo passivo.

7.3 Considerazioni finali

Il progetto Teseo ha dimostrato che la combinazione di un catalogo metadati open-source robusto, di pipeline di ingestion ben progettate e di strumenti AI governati tramite MCP può produrre un impatto operativo concreto e misurabile su una delle sfide più comuni delle organizzazioni data-driven moderne: la visibilità e la governabilità del patrimonio dati.

Il valore di Teseo non risiede soltanto nella tecnologia adottata, ma nell'approccio metodologico che ha guidato il progetto: progressività nell'adozione, modularità nell'architettura, attenzione alla reversibilità delle operazioni e integrazione tra dimensione tecnica e dimensione di governance. Questi principi rendono la piattaforma non solo funzionale nel contesto del progetto attuale, ma trasferibile come

framework a contesti aziendali diversi, indipendentemente dallo stack tecnologico specifico.

In prospettiva, l'evoluzione dell'ecosistema MCP e la crescente maturità dei modelli linguistici di grandi dimensioni suggeriscono che il paradigma *metadata-driven AI* adottato in Teseo diventerà sempre più rilevante nel panorama dei moderni data platform. La capacità di fornire agli agenti AI un contesto strutturato, governato e verificabile (piuttosto che un accesso grezzo ai dati) rappresenta una direzione architetturale che bilancia produttività e controllo, due requisiti difficili da soddisfare in contesti che puntano all'uso dell'AI direttamente nella dimensione del dato.

Bibliografia

- [1] IBM. *What is metadata management?* 2026. URL: <https://www.ibm.com/think/topics/metadata-management> (cit. alle pp. 1, 2, 74).
- [2] IBM. *What is data lineage?* 2026. URL: <https://www.ibm.com/think/topics/data-lineage> (cit. alle pp. 1, 3, 74, 78).
- [3] Snowflake. *Snowflake key concepts and architecture*. 2026. URL: <https://docs.snowflake.com/en/user-guide/intro-key-concepts> (cit. alle pp. 1, 3, 27, 76, 85).
- [4] dbt Labs. *What is dbt?* 2026. URL: <https://docs.getdbt.com/docs/introduction> (cit. alle pp. 1, 3, 77, 85).
- [5] DataHub Project. *Introduction to Metadata Ingestion*. 2026. URL: <https://docs.datahub.com/docs/metadata-ingestion> (cit. alle pp. 1, 18, 46, 48, 76, 77, 85).
- [6] Microsoft. *Data lineage in Power BI*. 2025. URL: <https://learn.microsoft.com/en-us/power-bi/collaborate-share/service-data-lineage> (cit. alle pp. 1, 4, 46, 77).
- [7] Model Context Protocol. *What is the Model Context Protocol (MCP)?* 2026. URL: <https://modelcontextprotocol.io/introduction> (cit. alle pp. 1, 4, 57, 61).
- [8] Target Reply. *Target Reply - Chi siamo*. 2026. URL: <https://www.reply.com/target-reply/it/> (cit. a p. 2).
- [9] DataHub Project. *Introduction - The #1 Open Source AI Data Catalog*. 2026. URL: <https://docs.datahub.com/docs/introduction> (cit. alle pp. 3, 10, 48, 50, 78).
- [10] DataHub Project. *DataHub Architecture Overview*. 2026. URL: <https://docs.datahub.com/docs/architecture/architecture> (cit. alle pp. 4, 8, 11).
- [11] Acryl Data. *Context Management Is the Missing Piece in the Agentic AI Puzzle*. 2026. URL: <https://datahub.com/blog/context-management-is-the-missing-piece-in-the-agentic-ai-puzzle/> (cit. alle pp. 4, 57, 81).

- [12] DataHub Project. *DataHub Components Overview*. 2026. URL: <https://docs.datahub.com/docs/components> (cit. alle pp. 8, 11, 18).
- [13] DataHub Project. *DataHub APIs and SDKs Overview*. 2026. URL: <https://docs.datahub.com/docs/api/datahub-apis> (cit. alle pp. 10, 22).
- [14] DataHub Project. *Metadata Model*. 2026. URL: <https://docs.datahub.com/docs/metadata-modeling/metadata-model> (cit. alle pp. 14–17, 50, 51).
- [15] DataHub Project. *Metadata Events (MCP, MCL, Platform Events)*. 2026. URL: <https://docs.datahub.com/docs/what/mxe> (cit. alle pp. 18, 19).
- [16] DataHub Project. *Metadata Ingestion Architecture*. 2026. URL: <https://docs.datahub.com/docs/architecture/metadata-ingestion> (cit. alle pp. 19, 21, 84).
- [17] DataHub Project. *DataHub GraphQL API Overview*. 2026. URL: <https://docs.datahub.com/docs/api/graphql/overview> (cit. alle pp. 22, 50, 56).
- [18] DataHub Project. *DataHub MCP Server*. 2026. URL: <https://docs.datahub.com/docs/features/feature-guides/mcp> (cit. alle pp. 46, 61, 62, 81).
- [19] Acryl Data. *DataHub MCP Server (GitHub repository)*. 2026. URL: <https://github.com/acryldata/mcp-server-datahub> (cit. alle pp. 46, 61, 62).
- [20] DataHub Project. *Databricks Source (Unity Catalog) - DataHub Ingestion*. 2026. URL: <https://docs.datahub.com/docs/generated/ingestion/sources/databricks> (cit. a p. 48).
- [21] Vijay Khatri e Carol V. Brown. «Designing Data Governance». In: *Communications of the ACM* 53.1 (2010), pp. 148–152. DOI: 10.1145/1629175.1629210 (cit. alle pp. 50, 70, 84, 86).
- [22] IBM. *What is data governance?* 2026. URL: <https://www.ibm.com/think/topics/data-governance> (cit. alle pp. 50, 70, 81, 84).
- [23] DataHub Project. *Assertion (DataHub Metamodel Entity)*. 2026. URL: <https://docs.datahub.com/docs/generated/metamodel/entities/assertion> (cit. alle pp. 56, 80).
- [24] DataHub Project. *Incident (DataHub Metamodel Entity)*. 2026. URL: <https://docs.datahub.com/docs/generated/metamodel/entities/incident> (cit. alle pp. 56, 80).
- [25] Model Context Protocol. *Model Context Protocol Specification (latest)*. 2026. URL: <https://modelcontextprotocol.io/specification/latest> (cit. alle pp. 57–60, 84, 86).
- [26] Anthropic. *Introducing the Model Context Protocol*. 2024. URL: <https://www.anthropic.com/news/model-context-protocol> (cit. alle pp. 57, 61).

- [27] Model Context Protocol. *Architecture overview*. 2026. URL: <https://modelcontextprotocol.io/docs/learn/architecture> (cit. alle pp. 58–60).
- [28] Model Context Protocol. *Example Clients*. 2026. URL: <https://modelcontextprotocol.io/clients> (cit. a p. 58).
- [29] Anthropic. *Building effective agents*. 2024. URL: <https://www.anthropic.com/research/building-effective-agents> (cit. alle pp. 71, 80, 84, 86).